



Research paper

A stable deep adversarial learning approach for geological facies generation

Ferdinand Bhavsar^{*}, Nicolas Desassis, Fabien Ors, Thomas Romary

Mines Paris, PSL University, Centre for geosciences and geoengineering, 77300 Fontainebleau, France

ARTICLE INFO

Keywords:

Deep learning
Generative adversarial networks
Simulations
Conditioning

ABSTRACT

The simulation of geological facies in an unobservable volume is essential in various geoscience applications. Given the complexity of the problem, deep generative learning is a promising approach to overcome the limitations of traditional geostatistical simulation models, in particular their lack of physical realism. This research aims to investigate the application of generative adversarial networks and deep variational inference for conditionally simulating channelized reservoir in underground volumes. In this paper, we review the generative deep learning approaches, in particular the adversarial ones and the stabilization techniques that aim to facilitate their training. We also study the problem of conditioning deep learning models to observations through a variational Bayes approach, comparing a conditional neural network model to a Gaussian mixture model. The proposed approach is tested on 2D and 3D simulations generated by the stochastic process-based model Flumy. Morphological metrics are utilized to compare our proposed method with earlier iterations of generative adversarial networks. The results indicate that by utilizing recent stabilization techniques, generative adversarial networks can efficiently sample complex target data distributions.

1. Introduction

The spatial distribution of lithofacies in the subsurface is needed in a wide range of geoscientific applications such as petroleum engineering (Caers, 2005), hydro-geology (Kitanidis, 1997), geophysics (Linde et al., 2015), in situ recovery of minerals (Langanay et al., 2021), geothermal resources assessment (Focaccia et al., 2016), carbon dioxide storage (Zhong and Carr, 2019), where it is required as an input for the fluid flow simulations of physico-chemical processes. As the information about these properties is generally scarce and uncertain, coming from a few well logs or core drillings (“hard” data), indirect geophysical data or geological qualitative information about the area (“soft” data), geoscientists often resort to probabilistic models to account for possible scenarios and characterize the related uncertainties. These models need to meet several conditions to be useful in practice: they need to be fast to generate a great number of geological facies simulations, honour the available data and exhibit geological consistency. They also necessitate being easily embedded into a data assimilation framework to incorporate other sources of data, e.g. fluid flow simulation results.

Geostatistics has aimed for decades to develop such models (Chilès and Delfiner, 2012). Object based or boolean models (Chiu et al., 2013), initially proposed to describe the distribution of grains and pores on a microscopic scale in reservoir rocks (Matheron, 1966) then materials (Jeulin, 2021), have been used at a macroscopic scale to

characterize the main geological bodies, such as channels and lenses in a reservoir (Dubrule, 1989). While powerful algorithms have been developed to generate conditional realizations of these models (Lantuéjoul, 2002), the lack of flexibility induced by the parameterization of the objects makes them difficult to handle and their geological realism questionable. Later on, plurigaussian models have been proposed (Matheron et al., 1987; Armstrong et al., 2001) to allow for more flexibility in the simulated bodies. The conditioning is made easy through the Gibbs sampler (Geman and Geman, 1984) but the inference of the model is not straightforward. In particular, the separation of the variability offered by the model between the facies proportions and the underlying Gaussian random function is not unique and hence must be cautiously carried out (Armstrong et al., 2001). Moreover, since the structure of the variability is driven by that of the latter two components, the realizations may fail to reproduce complex geological patterns. At the beginning of the century, the multiple-point approach (Strebel, 2002) arose as a potential method to generate geologically realistic conditional realizations. Based on the reproduction of features found in a training image, this method allows to reproduce virtually any texture and has been successfully applied to a large variety of problems (Mariethoz and Caers, 2014).

Meanwhile, sedimentologists have developed numerical models that attempt to reproduce the physics of the sediment deposition processes (Jacod and Joathon, 1971). As an example of such models,

^{*} Corresponding author.

E-mail address: ferdinand.bhavsar@gmail.com (F. Bhavsar).

the Flumy Model (Flumy-Userguide, 2022) generates 3D sedimentary deposits, taking into account processes such as avulsion, aggradation and migration. Due to their genetic nature, these models simulate realistic reservoirs. However, the generation is computer intensive. Furthermore, there exist, at the moment, no satisfactory way to condition the realizations to hard data (Bubnova, 2018; Troncoso, 2022).

Generative adversarial networks are a class of deep learning algorithms (Goodfellow et al., 2014) that exhibit great results in learning to generate fake realizations of complex multivariable distributions hardly distinguishable from real realizations (Arjovsky and Bottou, 2017) and have been applied in generating realistic pictures such as face images (Brock et al., 2019; Zhu et al., 2020) and art images (Karras et al., 2019). These models can be, despite these good results, hard to train, as discussed in Arjovsky and Bottou (2017) and research to improve the GAN method is still ongoing. Current improvements have come in the form of changing the architecture to generate images at different resolutions (Karras et al., 2017; Karnewar et al., 2019) or to use a Wasserstein loss (Arjovsky et al., 2017; Arjovsky and Bottou, 2017).

Given their seemingly limitless possibilities, geoscientists are using these models increasingly. One of the first successes in the domain was the reconstruction of porous media, using a deep convolutional variant of the algorithm trained on segmented volumetric images of porous media (Mosser et al., 2017). Reproduction of semi-straight and meandering channelized structures was done with a convolutional Wasserstein GAN model (Chan and Elsheikh, 2019). Later research showed the models could do it while having the flexibility to generate conditional simulations (Dupont et al., 2018). This method was extended to 3D volumes with a modified contextual loss (Zhang et al., 2021). Another method of conditioning applied Bayesian conditions, making use of Markov chain Monte Carlo (MCMC) methods to sample the posterior distribution. This method was successfully used with Variational Auto-Encoders models (Laloy et al., 2017) and later GANs (Laloy et al., 2018). Additionally, the progressive growing of GAN method also demonstrated good performances when used to generate geological facies (Song et al., 2021).

The main objective of this work is to develop a deep generative model capable of generating subsurface sedimentary facies simulations. These simulations must be realistic in the sense that they exhibit the same properties as the original dataset. Second, we want to be able to condition the model to observations. We present methods that aim to overcome the limitations of traditional geostatistical techniques and numerical process-based models. Throughout this paper, we offer a comprehensive overview of these methods, providing both theoretical background and practical guidelines. The main contributions of this paper are: (i) a stable generative architecture that combines recent development in the stabilization of GANs, namely the multiscale architecture trained with the Wasserstein loss with adapted activation functions; (ii) a new conditioning method to generate simulations given observations. This new conditioning method leverages a variational Bayes approach, approximating the conditional posterior using a Gaussian mixture model. Additionally, we propose a Conditional Generative Adversarial Network (CGAN) framework to modulate the simulated realizations according to different input parameter values.

We first introduce deep adversarial networks, before expanding on their improved versions and the architecture details for non-conditional generation. We then present two variational approaches to solve the conditioning problem, one from earlier literature using a neural network (Chan and Elsheikh, 2019), and a new method using a Gaussian Mixture. Both of which can be used in 2D and 3D. In the last section, we apply this model to generate Flumy-like stationary realizations of channelized reservoir in 2D and 3D, conditioning them to punctual data values, as well as showing the applicability of Conditional GANs (CGANs) to parameterized simulations, before concluding.

2. Stable stationary GAN

In the past few years, Generative Adversarial networks (Goodfellow et al., 2014), a sub-class of deep generative learning models, have been popular for being able to learn the approximation of complex, high-dimensional probability distribution using a large set of realizations thereof. These methods have seen a great amount of success in the generation of images such as pictures of faces and art (Arjovsky and Bottou, 2017; Brock et al., 2019; Zhu et al., 2020; Karras et al., 2019). Deep generative models have also increasingly been used to analyse spatial or spatio-temporal phenomena, making use of the flexibility of neural networks to learn complex data distributions (Mosser et al., 2017; Chan and Elsheikh, 2019; Laloy et al., 2018; Sun et al., 2023a; Liu et al., 2022; Zhang et al., 2022; Chen et al., 2022).

Generative Adversarial networks (GAN) consist of two separate neural networks, called the discriminator and the generator. These networks are trained by competing against each other. The goal is for the generator to sample synthetic data that are indistinguishable from training data, while the discriminator tries to distinguish between real and synthetic data. The generator produces synthetic data x by sampling from a Gaussian prior distribution and transforming it through a function $G(z, \theta^G)$, where z is a realization of the input distribution and θ^G are the parameters of the generator. Through G , the model thus defines the synthetic data distribution p_θ . The generator tries to fool the discriminator into asserting its generated examples as real. The discriminator assigns a probability value to each datum it receives as an input. Formally, it defines a family of functions $D(x, \theta^D)$ from the realization space to the interval $[0, 1]$, where x is a data point and θ^D are the parameters of the discriminator. The closer $D(x, \theta^D)$ is to 1, the higher the probability that x was sampled from the training data distribution $p_r(x)$. A sigmoid function usually serves as its last activation function to ensure that the output of the discriminator is a valid probability.

The models are trained as a zero-sum game, in which the payoff of the discriminator is denoted $v(\theta^G, \theta^D)$ and that of the generator is $-v(\theta^G, \theta^D)$. Both models will attempt to maximize their own payoff, meaning that the optimal model for both will optimize the following min-max equation:

$$\arg \min_{\theta^G} \arg \max_{\theta^D} v(\theta^G, \theta^D) = \arg \min_{\theta^G} \arg \max_{\theta^D} \mathbb{E}_{x \sim p_r} \log D(x) + \mathbb{E}_{x \sim p_\theta} \log(1 - D(x)) \quad (1)$$

Despite the initial successes, early literature authors identified how strenuous it is to train GANs using the original architecture (Goodfellow et al., 2014), with the need for extensive hyper-parameters tuning (Arjovsky et al., 2017; Arjovsky and Bottou, 2017). The models may suffer from the vanishing gradient phenomenon (Hochreiter, 1991). Another suspected cause is the demonstrated fact that support of p_r lies on low-dimensional manifolds (Narayanan and Mitter, 2010), that p_θ is unlikely to cover at the early steps of the training. This makes the loss function saturated, which also causes the gradient to vanish.

A proposed strategy to make the support overlap more likely concentrate on reducing the effect of the curse of dimensionality. Karras et al. (2017) tried to stabilize the outputs of the generator and speed-up convergence by making a GAN model progressively grow. They built more and more detailed image generators, starting with a 4×4 generator, and then adding layers during training. Intuitively, this helps with the overlap problem, since the distributions will match first on lower resolutions where the overlap is more likely to be non-negligible. Progressively Growing GAN have already been used for Geological facies simulation, both in 2D and 3D (Song et al., 2022).

Multi-Scale GANs (Karnewar et al., 2019) relies on similar theoretical ideas to the progressive growing of GAN (Karras et al., 2017), but differ from the original method as the models do not grow throughout training. The model stays the same and learn at different scales from

start to end. Thus the generator simulates the realization at different scales while the discriminator assesses its realism at each scale. The real and generated realizations at different scales are concatenated through these connections to the activation features of the discriminator at the corresponding scale.

The advantage over the progressive growing of GANs is a simpler implementation, as well as allowing gradient circulation in the model at all scales all along the training, preventing the vanishing gradient problem.

Instead of making the overlap more likely, another popular method avoids the problem completely by resorting to a different loss function. The Wasserstein-1 distance, a distance measure between two probability distributions, has been proposed in Arjovsky et al. (2017). This loss can be computed using its dual form named Kantorovich–Rubinstein dual (see Appendix C.1 for details). This approach requires however the discriminator to be 1-Lipschitz (Appendix C.1). To enforce this condition, we use a Spectral Normalization (Miyato et al., 2018):

$$W_{SN} = \frac{W}{\sigma(W)} \quad (2)$$

where $\sigma(W)$ is the spectral norm of parameters W .

We also change the activation functions following the recommendations made by Anil et al. (2018). We use non-monotonic, 1-Lipschitz bounded functions or gradient preserving functions such as GroupSort, Fpflu (Zhu et al., 2021) and the Swish activation functions (Ramachandran et al., 2018) normalized to have unit gradient norm. Furthermore, to ensure that our model is entirely translation invariant, as we model stationary processes, our models are entirely convolutional. This model's design allows for the generation of images of variable size by adjusting the input noise. This aspect is similar to previous work on stationary GANs (Jetchev et al., 2016; Long et al., 2015; Laloy et al., 2018). The generation of non-stationary phenomena is out of scope for this work, however the model can be modified for such a purpose, such as in Abdellatif et al. (2023).

3. Variational Bayes conditioning

Our objectives are not only to propose a stable deep generative model that simulates realistic reservoirs, but also a way to condition this generator to observations. We describe here two competing approaches based on a variational Bayes framework.

Once trained to optimality, the GAN framework provides a generator G that can simulate new realizations with the same properties as the training dataset, as such we can consider that using the generator samples x . Applications, in resource prospecting for example, require sampling realizations knowing partial information about x . In geology, this partial information can come from boreholes or seismic imaging. We denote the partial information x^* . Its conditional distribution with respect to x , also called observation process, is modelled by specifying the conditional density $p(x^*|x)$. When generating realizations through the generator, $x = G(z)$, with z the input of the generator, we can instead consider $p(x^*|z)$, where x^* is a set of N observations. We recall that z is a standard Gaussian random vector with independent components, that defines the prior distribution $p(z)$. Therefore, the conditioning problem consists in obtaining the distribution $p(z|x^*)$ by using the Bayes formula:

$$p(z|x^*) = \frac{p(x^*|z)p(z)}{p(x^*)} \quad (3)$$

$$p(x^*) = \int p(x^*|z)p(z)dz \quad (4)$$

The computation cost of this integral is exponential in the dimension of the problem. Consequently, instead of computing it exactly, we will use methods that allow us to ignore the normalizing constant. Monte-Carlo Markov Chains (MCMC) methods are usually used to sample from such a distribution. Previous methods have successfully applied

MCMC complementary to GANs for geological simulations to sample conditional distributions (Laloy et al., 2018). However, they can be slow to converge and may encounter difficulties to explore several posterior modes. Ensemble methods have also been successfully applied to condition geological models (Canchumuni et al., 2019; Fossum et al., 2022, 2024).

Variational methods change the sampling problem into an optimization problem, which is solved through gradient descent. To do so, we limit our research to a family of parametric distributions Q , defined on the same probability space as $p(z|x^*)$, the real latent distribution. We search for the member of this family that minimizes the Kullback–Leibler Divergence between itself and the real distribution:

$$q_\theta^*(z|x^*) = \arg \min_{q_\theta(z|x^*) \in Q} D_{KL}(q_\theta(z|x^*) \parallel p(z|x^*)) \quad (5)$$

An inference model with parameters θ is used to define the parametric distribution q_θ . See Fig. 1 for an overview of the conditioning method. The inference model can be any kind of parametric model that approximate a distribution, we simply use the term inference model as a placeholder. This workflow for the conditioning problem allows the use of common optimization methods, to estimate the parameters θ of the inference model. Moreover, once trained, the variational approximation can be used to sample new realizations of the posterior distribution, allowing us to generate any number of conditional realizations.

3.1. Neural network inference model

In Chan and Elsheikh (2017) and Chan and Elsheikh (2019), the authors use an inference neural network I_θ , such that the family of parametric distribution Q is described by the set of all the functions defined by the neural network I applied to a given set of random variables. According to the Universal Approximation Theorem (Pinkus, 1999; Kidger and Lyons, 2019), by choosing a neural network deep and/or wide enough, we can learn an approximation of the true posterior distribution $p(z|x^*)$.

The neural network defines the parametric distribution q_θ by transforming $\hat{z}$, a standardized Gaussian vector with independent components, into $z|x^*$. The model is a simple Multi-Layer Perceptron, with dense layers and Selu activation functions (Klambauer et al., 2017). The input latent space and output space are of the same dimension.

The inference neural network I_θ is parameterized by θ , which are the weights of the linear layers. It needs to be trained such that its weights θ are optimal. The optimal weights θ^* minimize the Kullback–Leibler divergence between the parametric distribution and the real posterior distribution:

$$\begin{aligned} \theta^* &= \arg \min_{\theta} D_{KL}(q_\theta(z|x^*) \parallel p(z|x^*)) \\ &= \arg \min_{\theta} \mathbb{E}_{z \sim q_\theta} (\log p(z|x^*)) - \mathbb{E}_{z \sim q_\theta} (\log q_\theta(z|x^*)) \\ &= \arg \min_{\theta} \mathbf{H}(q_\theta, p) - \mathbf{H}(q_\theta), \end{aligned} \quad (6)$$

where $\mathbf{H}(q_\theta, p)$ is the negative cross-entropy between the training data distribution p and the variational distribution q_θ . If we have C different possible values for x , then the negative cross-entropy is defined as follows:

$$\sum_i^C p(x_i) \log(q_\theta(x_i)) \quad (7)$$

In our application, the observations x^* are vertical wells, which are locations in the realization where the category of the facies is known. If we assume that observations are independent realizations of multinomial variables with probabilities given by the generator, we can compute the negative cross-entropy by using the multi-class cross-entropy. $\mathbf{H}(q_\theta)$ is the Shannon Entropy of the probability distribution q_θ . Hence, minimizing Eq. (6) amounts to minimizing the cross-entropy

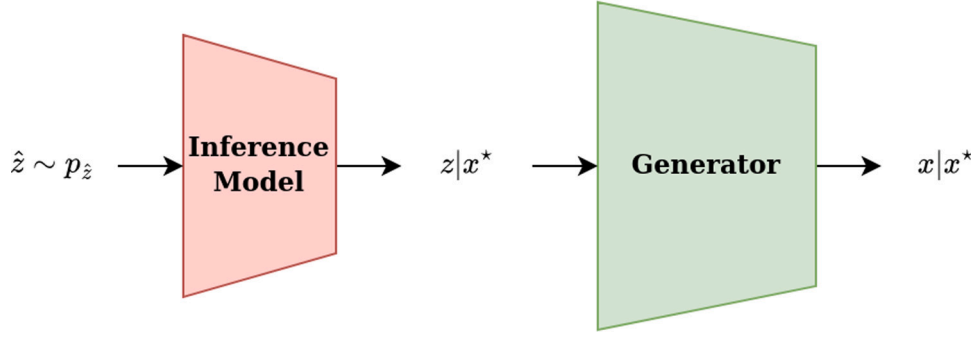


Fig. 1. Following the workflow from (Chan and Elsheikh, 2019), we use a secondary model that will transform a random vector $\hat{z}$ into the conditioned variable $z|x^*$.

between the two distributions while keeping the entropy of the variational distribution high. The latter ensures that the model maximizes the variability of the posterior distribution.

Since we consider that the observations are independent from a multinomial distribution and $p(z)$ is the multivariate standard normal distribution, we can use Bayes' rule to compute $\mathbf{H}(q_\theta, p)$ as follows:

$$\begin{aligned} \mathbf{H}(q_\theta, p) &= \mathbb{E}_{z \sim q_\theta} [-\log p(z|x^*)] \\ &= \mathbb{E}_{z \sim q_\theta} \left[-\sum_{i=1}^N \log p(x_i^*|z) + \frac{1}{2} \|z\|^2 \right] \end{aligned} \quad (8)$$

with N the number of individual conditioning observation points $x_{1 \leq i \leq N}^*$. The expectation is intractable, because we can only sample $p(x_i^*|z)$ through the generator. As such we use a Monte-Carlo estimator over a batch of M generated samples $z_m = I(\hat{z})$ to estimate it:

$$\mathbf{H}(q_\theta, p) = \frac{1}{M} \sum_{m=1}^M \left[-\sum_{i=1}^N \log p(x_i^*|z_m) + \frac{1}{2} \|z_m\|^2 \right] \quad (9)$$

The second term of (6), namely the entropy $\mathbf{H}(q_\theta)$ requires also an approximation since we have access to q_θ only through its realizations. $\mathbf{H}(q_\theta)$ can be approximated through the Kozachenko–Leonenko estimator (L.F. Kozachenko, 1987), see Appendix C.4 for more details. The approximate loss function is hence defined as follows:

$$\begin{aligned} D_{KL}(q_\theta(z|x^*) \parallel p(z|x^*)) \\ \approx -\frac{d}{M} \sum_{m=1}^M \log(d_{knn}(z_m)) + \frac{1}{M} \sum_{m=1}^M \left[-\sum_{i=1}^N \log p(x_i^*|z_m) + \frac{1}{2} \|z_m\|^2 \right] \end{aligned} \quad (10)$$

with $d_{knn}(z_m)$ the distance from sample z_m to its k th neighbour. Following (Chan and Elsheikh, 2019), we set $k = \sqrt{M}$. We use this approximation of the divergence to train the model using Algorithm 1.

Algorithm 1 Training neural network conditioning

- 1: N observations x_0 , $M \in \mathbb{N}$ the batch size
 - 2: **repeat**
 - 3: Draw M samples from $\hat{z} \sim \mathcal{N}(0, I_d)$
 - 4: Apply inference model: $z = I_\theta(\hat{z})$ $\triangleright z \sim q_\theta(z|x^*)$
 - 5: Apply generator neural network: $x = G(z)$ $\triangleright x \sim q_\theta(x|x^*)$
 - 6: Update I_θ using gradient step on $D_{KL}(q_\theta(z|x^*) \parallel p(z|x^*))$ $\triangleright$
 - See Eq. (10)
 - 7: **until** Convergence
-

3.2. Gaussian mixture inference model

The previous approach may suffer from mode collapse, which is an optimization error where the algorithm only approximates one mode

of a multimodal distribution. Indeed, the Kullback–Leibler divergence is blind to isolated modes (Wenliang and Kanagawa, 2020). We propose to alleviate this problem by using a Gaussian mixture inference model in the variational approach.

We choose a number of Gaussian components K . Each component $i = 1, \dots, K$ is defined by its mean μ_i and its covariance matrix Σ_i , as well as a non-negative weight π_i . The weights π_i sum to 1. The parametric distribution is then defined as $q_\theta(z) = \sum_{i=1}^K \pi_i f_{\mu_i, \Sigma_i}(z)$, where $f_{\mu, \Sigma}$ is the Gaussian density with mean μ and covariance Σ .

Similarly to the inference network method, a standardized Gaussian vector with independent components $\hat{z}$ is sampled. Then a random component of the mixture i is chosen using the mixture weights. The mean and covariance of the chosen component are finally applied to the initial standard Gaussian random vector with independent components:

$$z = \mu_i + \Sigma_i^{1/2} \times \hat{z}, \quad (11)$$

where $\Sigma_i^{1/2}$ is e.g. the Cholesky decomposition of Σ_i .

To train the parameters of the components we use again the Kullback–Leibler divergence as an objective function. This means that we cannot guarantee that it will find every modes of the distribution. However, in our experiments, we have found that by selecting enough components in the mixture and initializing them randomly, this method exhibits a greater degree of diversity than the neural network inference model. Choosing a large number of components increases the complexity of the model, as well as the time it takes to train it.

The Kullback–Leibler divergence is decomposed in the same manner as above. We keep θ as the notation for parameters, noting that $\theta_i = (\pi_i, \Sigma_i, \mu_i)$ and $\theta = (\theta_1, \theta_2, \dots, \theta_K)$. Since we are working with Gaussian Mixtures we can compute the entropy using the approximation proposed by Furuya et al. (2022):

$$\begin{aligned} \mathbf{H}(q_\theta) &\approx \tilde{\mathbf{H}}(q_\theta) \\ &= -\sum_{i=1}^K \pi_i \int \mathcal{N}(z|\mu_i, \Sigma_i) \log(\pi_i \mathcal{N}(z|\mu_i, \Sigma_i)) dz \\ &= \frac{m}{2} + \frac{m}{2} \log 2\pi + \frac{1}{2} \sum_{i=1}^K \pi_i \log |\Sigma_i| - \sum_{i=1}^K \pi_i \log \pi_i \end{aligned} \quad (12)$$

The cross-entropy is also weighted by the components used to generate the simulations of the current batch. During an iteration, we sample M generated Gaussian vectors z_m . Each z_m is generated by sampling one random mixture component with parameters (π_m, Σ_m, μ_m) . Finally, the Kullback–Leibler divergence is approximated as follows:

$$D_{KL}(q_\theta(z|x^*) \parallel p(z|x^*)) \approx -\tilde{\mathbf{H}}(q_\theta) + \frac{1}{M} \sum_{m=1}^M \pi_m \left[-\sum_{i=1}^N \log p(x_i^*|z_m) + \frac{1}{2} \|z_m\|^2 \right] \quad (13)$$

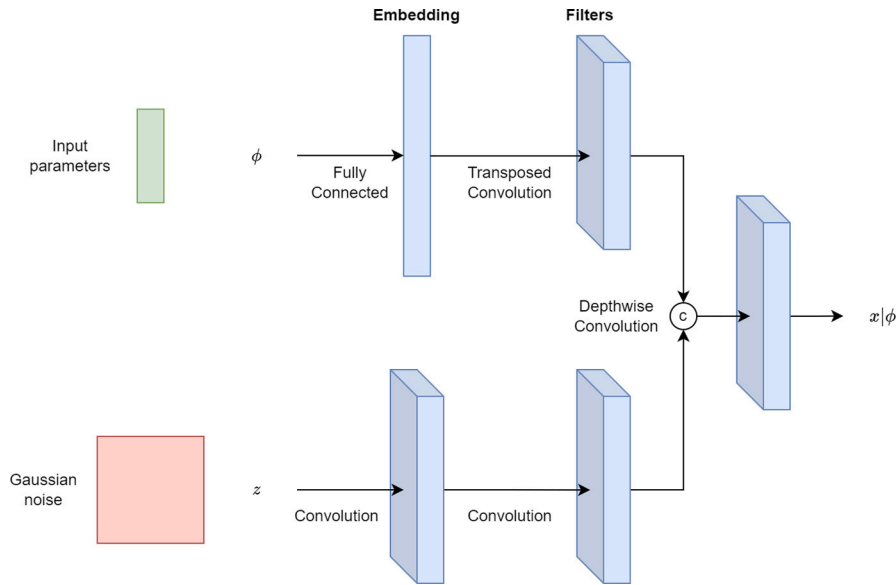


Fig. 2. Diagram of the stationary conditional generator, where the input parameters ϕ are two scalar values in a vector (Net to Gross and Sand Body Extension), which are then projected into an embedding space before computing convolution kernels that are used in the network.

We use this approximation of the divergence to train the model using Algorithm 2.

Algorithm 2 Training Gaussian mixture conditioning

```

1:  $N$  observations  $x_0, M \in \mathbb{N}$  the batch size,  $K \in \mathbb{N}$  the number of Gaussian mixture components
2: repeat
3:   Draw  $M$  samples  $\hat{z} \sim \mathcal{N}(0, I_d)$ 
4:   for  $i \leftarrow 1, M$  do  $\triangleright$  Note that the for loop is in practice parallelized
5:     Draw mixture component  $\theta_k = (\pi_k, \Sigma_k, \mu_k)$  using weights  $(\pi_1, \dots, \pi_K)$ 
6:     Apply inference model:  $z_i = \mu_k + \Sigma_k^{1/2} \times \hat{z}_i \triangleright z_i \sim q_{\theta_k}(z_i|x^*)$ 
7:     Apply generator neural network:  $x_i = G(z_i) \triangleright x_i \sim q_{\theta_k}(x_i|x^*)$ 
8:   end for
9:   Update  $\theta = (\theta_1, \dots, \theta_K)$  using gradient step on  $D_{KL}(q_\theta(z|x^*) || p(z|x^*)) \triangleright$  See Eq. (13)
10: until Convergence

```

It should be noted that in this algorithm, we distribute the batch among the different components by performing a weighted sampling based on the weights $(\pi_1, \dots, \pi_K)$ from the previous step. This approach is adopted for efficiency in Monte Carlo estimation of the posterior. However, it is worth mentioning that other choices would also be valid. For instance, sampling each component equally could be another legitimate alternative.

4. Parameterized generative model

The GAN framework can be upgraded such that it can be parameterized with some extra information ϕ (Mirza and Osindero, 2014). In the original paper, ϕ is a set of complementary information about the desired simulation, such as a class labels or physical parameters. This complementary information is inputted to both the generator and the critic. As such, the generator learns how to generate a realization $x = G(z|\phi)$ and the critic learns to give a probability $D(x|\phi)$ that the realization is real and corresponds to parameters ϕ . Consequently, the loss function in this new framework becomes:

$$v(\theta^G, \theta^D) = \mathbb{E}_{x \sim p_r} \log D(x|\phi) + \mathbb{E}_{x \sim p_g} \log(1 - D(x|\phi)) \quad (14)$$

The problem with the original conditional GAN is that it is not translation invariant, because the architecture includes fully connected layers. Instead, we developed a stationary conditional GAN framework. Our approach does not change the output of linear layers but learns instead different convolution kernels for each pair of input parameters.

This is done by adding a linear layer that will transform the parameters to an embedding space, which is then given to a transposed convolution layer. The outputs of these two layers are convolution kernel $\mathcal{K}_\phi \in \mathbb{R}^{N \times N}$ that can be applied depth-wise to feature matrices. Thus, the convolution kernels $\mathcal{K}_\phi$ define a convolutional layer with weights dependent on input ϕ (see Fig. 2). These layers and their parameterized filters are present in the generator and the critic. Thus, both networks now have a second input in addition to their normal input. These two layers are trained alongside the generator and critic with the same gradient.

5. Application

In this section, we apply the generative models described earlier, to reproduce reservoir model simulations generated with Flumy, a process-based model for meandering channelized reservoir, similarly to Sun et al. (2023b). Every model and metric used for our experiments are coded in Python and the Tensorflow library is used to design the deep learning models. The models are trained with Graphical Processing Units from Google Colaboratory resources. In order to evaluate the quality of the simulations generated by the different GAN architectures tested, we compute and compare morphological properties such as facies proportions and connected components size distribution between GAN simulations and training dataset. The gstlearn (gstlearn, 2023) Python package from MINES Paris is used for this purpose. We then test the best model on the task of conditioning.

5.1. Flumy

Flumy is a stochastic process-based model used to simulate the geometry of heterogeneous reservoirs generated by meandering channelized systems in both fluvial and turbidites environments (Lopez, 2003a,b; Flumy-Userguide, 2022; Grimaud et al., 2022). Flumy combines first-order hydraulic equations to track the evolution of channels and stochastic algorithms simulating chute cutoff, levee breaching, avulsions and associated deposits. See Fig. 3 for a visual overview

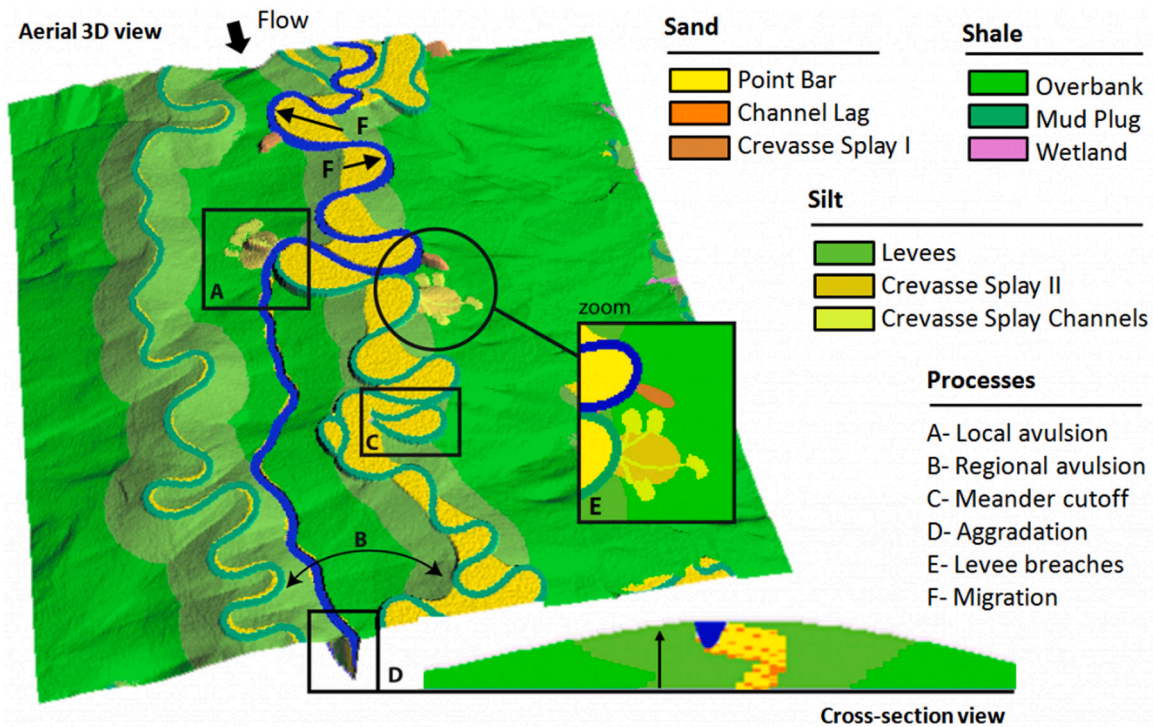


Fig. 3. Overview of Flumy's main simulated processes for Fluvial systems (Flumy-Userguide, 2022).

of Flumy's simulated processes. Flumy generates 3D reservoir models following a user-defined multi-sequence scenario.

Conditioning to existing subsurface datasets, while possible (Bubnova, 2018; Troncoso, 2022), is difficult and the method lacks of flexibility. Furthermore, inference computational cost is high, making it a choke-point for workflows where realizations must be simulated many times.

5.2. Experimental setup

Experiments are conducted using a dataset generated using the Flumy model. See Appendix F for more details on the Flumy parameters used to generate the training dataset.

The 2D GAN models are trained using a dataset containing 3000 horizontal slices extracted from 150 Flumy reference simulations having 4 facies (Channel Lag, Point Bar, Levee and Overbank) at a 10 m resolution (2D realization size is 64×128 pixels). The 3D GAN models are trained using a dataset containing 3000 3D blocks extracted from 1500 Flumy reference simulations with a vertical discretization step of 0.5 m, having the same 4 facies at a 50 m horizontal resolution (3D realization size is $16 \times 32 \times 64$ pixels). Realizations are represented as matrices, and at each position, the type of geological facies is a categorical vector of facies indicators. Consequently, in our dataset, there are only 4 different values a pixel can take (i.e. $[1,0,0,0]$, $[0,1,0,0]$, $[0,0,1,0]$ or $[0,0,0,1]$).

Later in the application we test a CGAN model (Mirza and Osindero, 2014) for parameterized simulations. For this part of the application, we use other Flumy simulations which are variations around the default Fluvial scenario (see Flumy-Userguide, 2022). Each Flumy simulation is tuned by setting a specific value of the two following key parameters: the required proportion of sand (aka. Net to Gross or NG) and the required mean Sand Body Extension Index (ISBX). Then, the 2D CGAN model is trained using a dataset of horizontal slices extracted from the different parameterized Flumy simulations with 9 facies (Channel Lag, Point Bar, Sand Plug, Crevasse Splay I, Crevasse Splay II Channels, Crevasse Splay II, Levee, Overbank, Mud Plug) at a 10 m horizontal resolution (2D realization size is 64×128 pixels).

5.3. Channelized reservoir simulations results

5.3.1. 2D

Initially, we test the Multi-Scale Wasserstein GAN with Spectral Normalization and GroupSort, whose architecture is given in Appendix A on 2D simulations. In these tests, the generator architecture stays mostly the same, with two (3,3) convolution operations at each level, LeakyReLU activation functions, and a final Softmax activation function. Furthermore, multi-scale generators require a pixelwise skip-connection at each level too. The critic architecture reflects the one of the generator, with two (3,3) convolutions operations at each levels. To reflect the skip connection in the generator, our critic returns a score at each level, with a (3,3) convolution from which the mean value is our level score. The GroupSort operation is used everywhere after every convolution in the critic, with the exception of the outputs convolution.

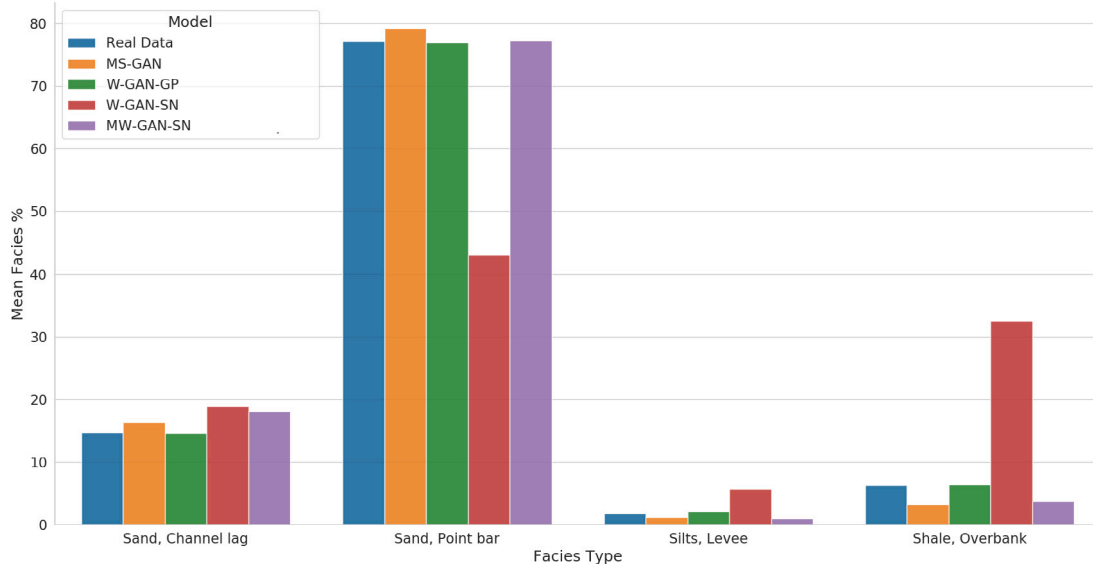
For comparison purposes, we also train a Multi-Scale GAN with the vanilla GAN loss, and only the addition of a Gaussian noise to stabilize it. We also train two vanilla Wasserstein GAN, one model with the GroupSort activation and Spectral Normalization, and the second model with LeakyReLU and Gradient Penalty. The fourth architecture is similar to the one used in Liu et al. (2023), with the exception that they use Attention layers (Vaswani et al., 2017), and did not use the GroupSort activation function. The detail of these architectures are summarized in Table 1.

Given the subjective nature of human visual assessment, we chose to quantify the quality of the non-conditional simulations using morphological metrics calculated from 1000 simulations per model. The distribution of connected component sizes by facies for multiple 2D models is depicted in Fig. 5 and compared to the training data. A Wasserstein-1 loss is computed, between each of these distributions and the real distribution (Table 2). The mean proportion of each geological facies for the 2D models is also shown in Fig. 4. We can see on these figures that the distribution of smaller features ("Sand, Channel lag", "Silts, Levees" and "Shale, Overbank") are correctly predicted for most models except for the Wasserstein GAN with spectral normalization. However, the "Sand, Point bar" prediction has a very high distance

Table 1

Summary of the architecture details as well as the hyper-parameters used to train our showcased 2D models. Here “lr gen./disc.” refers to the two learning rates used respectively for the generator and the discriminator (or critic). The green row indicates our recommended architecture.

Name	Loss	Multiscale	Disc. Lipschitz constraint	Disc. activ. func.	Epochs	lr gen./disc.
MS-GAN	MinMax	✓	✗	LeakyReLU 0.2	100	$5e^{-4}/5e^{-4}$
W-GAN-GP	Wasserstein	✗	Gradient Penalty	LeakyReLU 0.2	150	$5e^{-4}/1e^{-4}$
W-GAN-SN	Wasserstein	✗	Spectral Norm.	Group Sort	200	$5e^{-3}/5e^{-4}$
MW-GAN-SN	Wasserstein	✓	Spectral Norm.	Group Sort	200	$5e^{-3}/5e^{-4}$

**Fig. 4.** Comparison of the proportions of different facies for our 2D models.**Table 2**

To compare the connected components size distribution in 2D, shown in Fig. 5, the Wasserstein-1 distance between the model and the training data distributions is computed. The green row is our recommended architecture.

Model	Sand, Channel lag	Sand, Point bar	Silts, Levee	Shale, Overbank
MS-GAN	0.64	126.64	0.39	2.67
W-GAN-GP	0.11	16.47	0.29	0.90
W-GAN-SN	0.75	143.33	1.39	6.09
MW-GAN-SN	0.97	14.21	0.33	1.68

value as seen in Table 2. We did not find any explanation for this phenomenon.

The metrics comparison for the different models shown in Figs. 5 and 4 suggest that models utilizing Wasserstein or Multi-Scale techniques are able to generate simulations with the same characteristic features as the input dataset. In Fig. 5, the distributions of the connected components size of each geological facies for different models are compared against training data. We can see from the comparison that our proposed method MW-GAN-SN is only the second best when reproducing the morphology of most facies except the “Sand, Point bar”. We acknowledge that in 2D, a simple model seems more efficient. However, when we started to generate 3D blocks (see next section), we were not able to reach the convergence while training any other model except the latter.

Furthermore, the model generates unique simulations which are not copies from the dataset. This can be seen in Fig. 6, where we visually compare generated realizations to their nearest neighbour, in the Euclidean space, from the Flumy dataset.

5.3.2. 3D

Once established the robustness of the 2D model, we extend the analysis to 3D simulations (Fig. 7). The architecture design of the 3D model is identical to 2D architecture, with the exception that the 2D

Table 3

Hyper-parameters used to train our showcased 3D model.

3D Model	MW-GAN-SN-NormSwish
Epochs	200
Learning rate generator/discriminator	$1e^{-3}/1e^{-3}$

convolutions becomes 3D convolutions and the activation function is a normalized Swish. The hyper-parameters used to train 3D models are showcased in Table 3.

We evaluate the 3D model using the same metrics as in 2D. Those are pictured in Figs. 9 and 8.

Training 3D models proved to be more challenging. Ultimately only a combination of Wasserstein with the non-monotonic, 1-Lipschitz and gradient preserving activation function GroupSort, and Multi-Scale techniques was successful. Both visual and morphological evaluations of the resulting 3D model demonstrate its ability to generate geological structures that are consistent with the input dataset. The visual evaluation, shown in Fig. 7, reveals geological patterns found both in the Flumy simulations and the GAN simulations, specifically, the “V” pattern of sand in vertical sections and the elongated sand channels in horizontal slices. We also can notice the small scattered shale lenses in both Flumy simulations and GAN simulations. This is a good sign

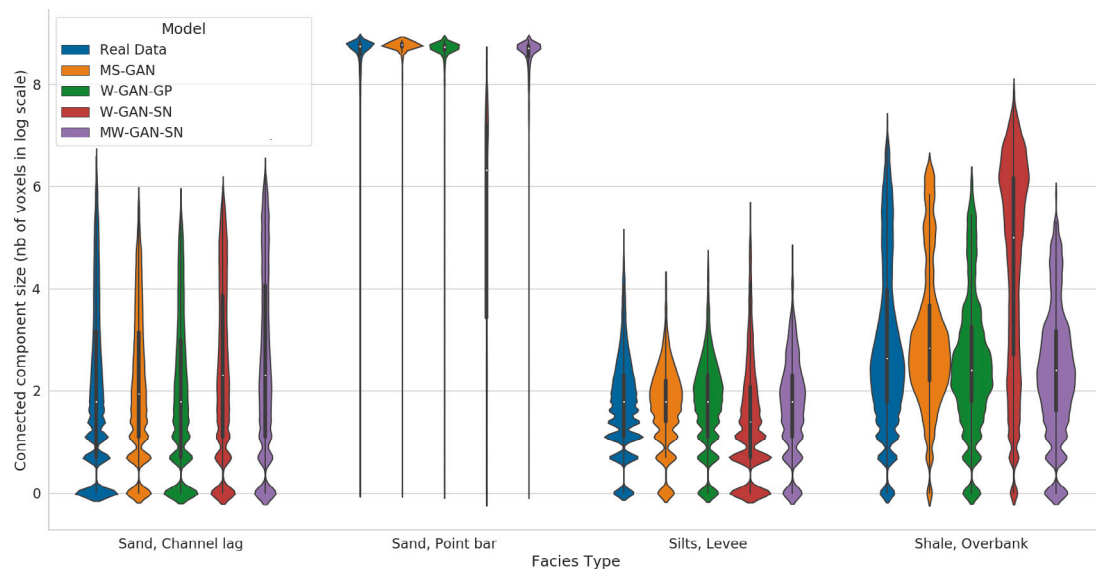


Fig. 5. Distribution of connected components size of each geological facies for different models against training data. To make visualization more relevant, each connected component size is itself weighted by its volume.

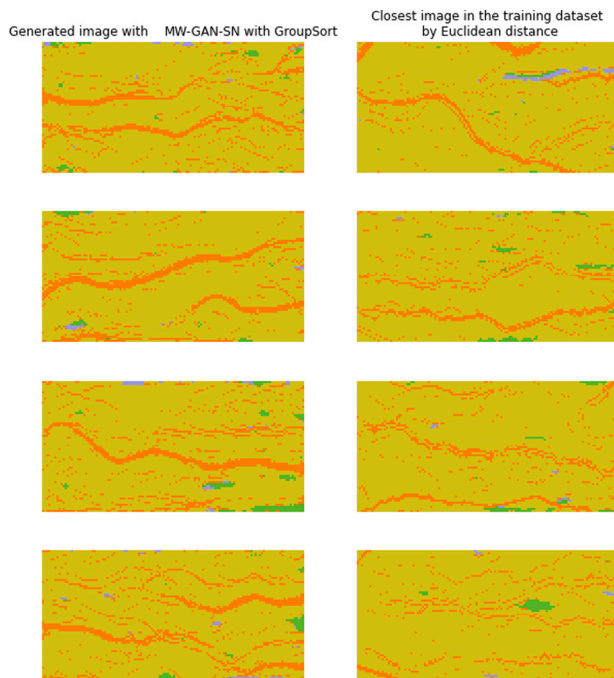


Fig. 6. Comparison of the generated realizations (on the left) to their nearest counterparts in the training dataset (on the right) using Euclidean distance.

that the models captures the features from the input dataset. However, we can notice that the generated sand channels are not as horizontally continuous as the ones generated by Flumy.

5.4. Parameterized model results

In order to capture the large range of possible geometries in channelized reservoir models, Flumy proposes two key parameters: the Net to Gross (sand proportion) and the sand bodies extension (lateral extension of point bars). Using these two parameters, the user can generate a large variety of different reservoirs (see [Flumy-Userguide, 2022](#)).

We evaluate here the CGAN approach described in Section 4. The dataset consists of 8000 Flumy realizations from multiple distributions, corresponding to different parameter values. The generator is given random parameter values within the range of the real parameters when trained, allowing it to interpolate between parameter values not present in the training data.

The simulations generated with Flumy using a range of ISBX and NG values used to train our CGAN experiments are pictured in [Fig. 10](#). [Fig. 11](#) displays simulations of the trained CGAN model. As can be seen from the above figures, the CGAN model can be modified to learn the relationship between parameters and output realizations. The generator learns to account for these parameters, making it a useful feature for those wishing to use it as a proxy for Flumy.

5.5. Conditional generation

5.5.1. 2D

Conditional generation can be tested by sampling Flumy simulations, randomly selecting vertical well locations and masking the rest. In 2D, wells are pixels in the realization where the category of facies is known. Similarly to the non-conditional methodology, we first test the method in two-dimension and only after in three-dimension. We evaluate how strictly the constraints are satisfied using the percentage of correctly generated values at the well. The average percentage of successfully predicted observations shown in [Fig. 13](#), seems to indicate that the neural network model is slightly better than the mixture of Gaussians models at honouring the conditioning observations (see [Fig. 12](#)).

However, assessing the extent to which the estimated posterior captures the variability of the real posterior is non-trivial. Instead, we judge how well it is captured by our network by comparing it to rejection sampling. We generate thousands of simulations with the non-conditional generator. We reject any realization that do not fulfil the given conditions. We then sample 2500 simulations using the two inference models. We finally compare the probability at each pixel of every facies category between the two methods. This is shown in [Fig. 14](#).

The results showcased highlight that the inference neural network does not seem to explore the whole domain of possible conditional realizations. In the top row, the probabilities of the first facies are only close to one in the immediate neighbourhood of the conditional

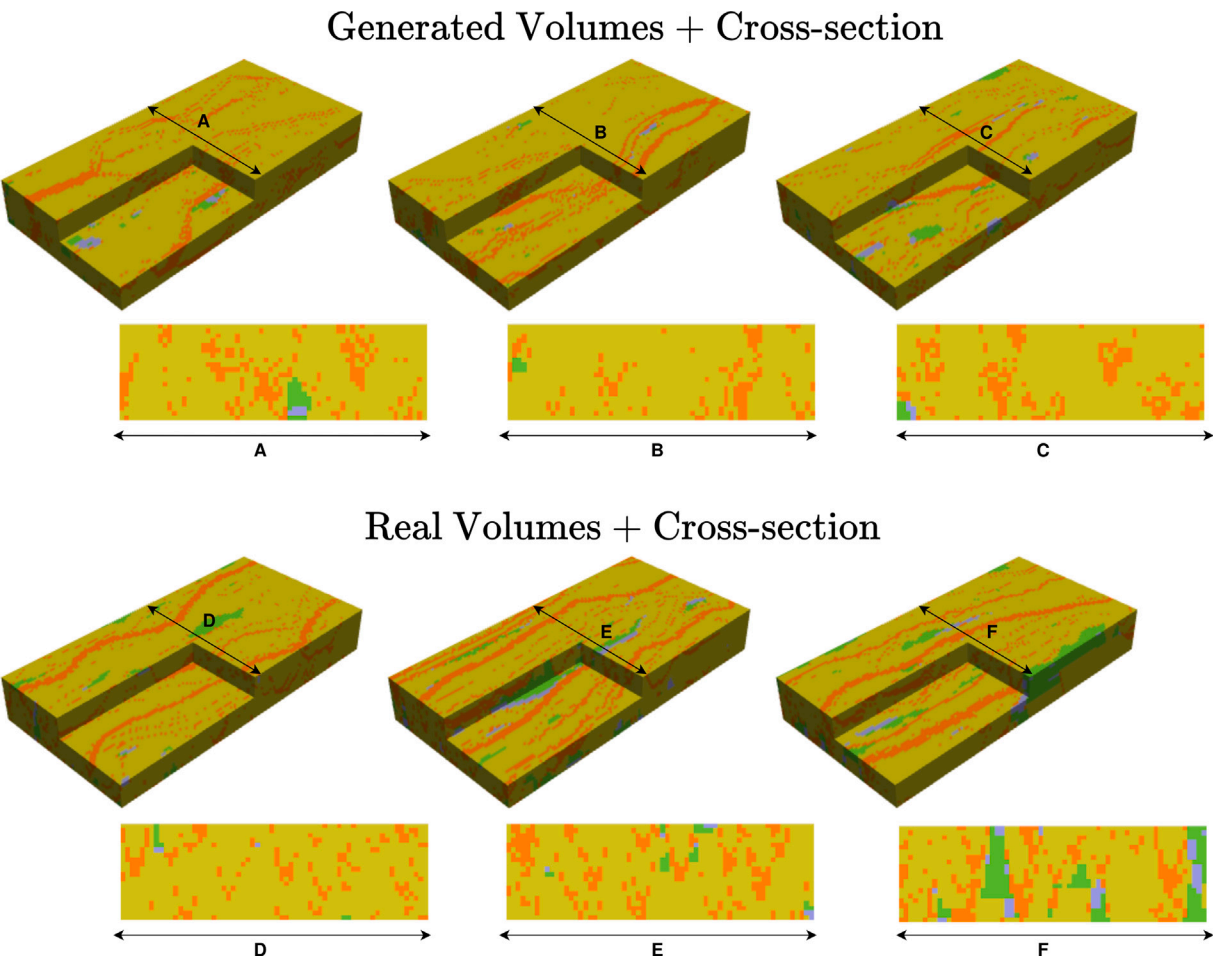


Fig. 7. Visual comparison of 3D volumes generated with our Multi-Scale Wasserstein GAN (top) and the Flumy 3D volume (bottom). Aerial views are accompanied by cross-section views.

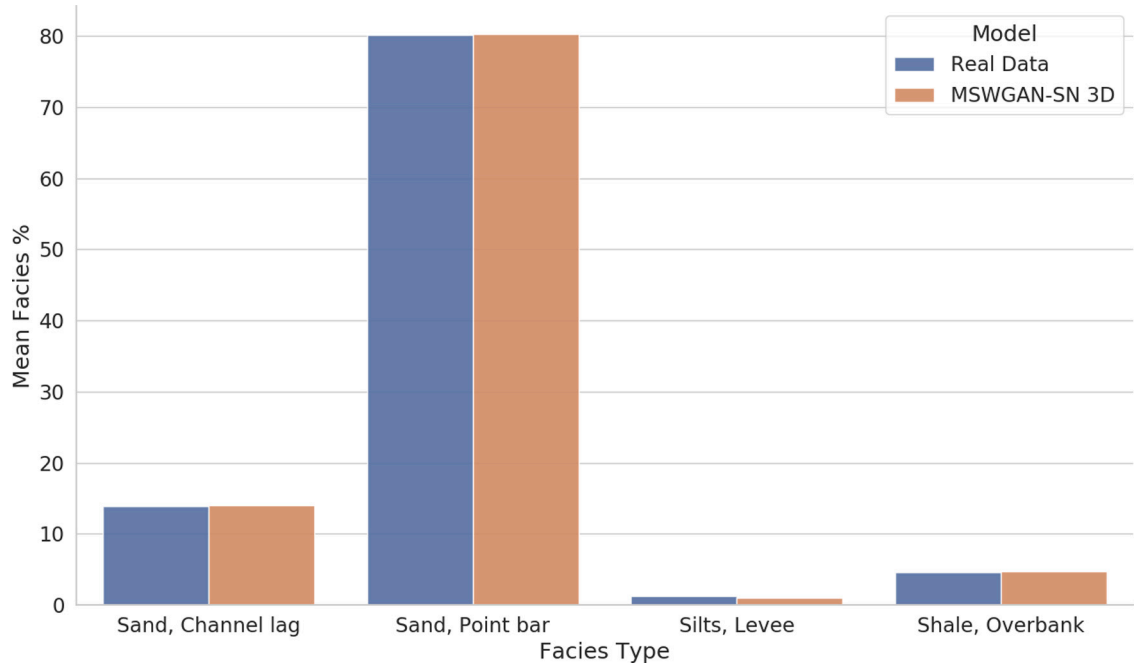


Fig. 8. Comparison of the proportion of different facies for our models.

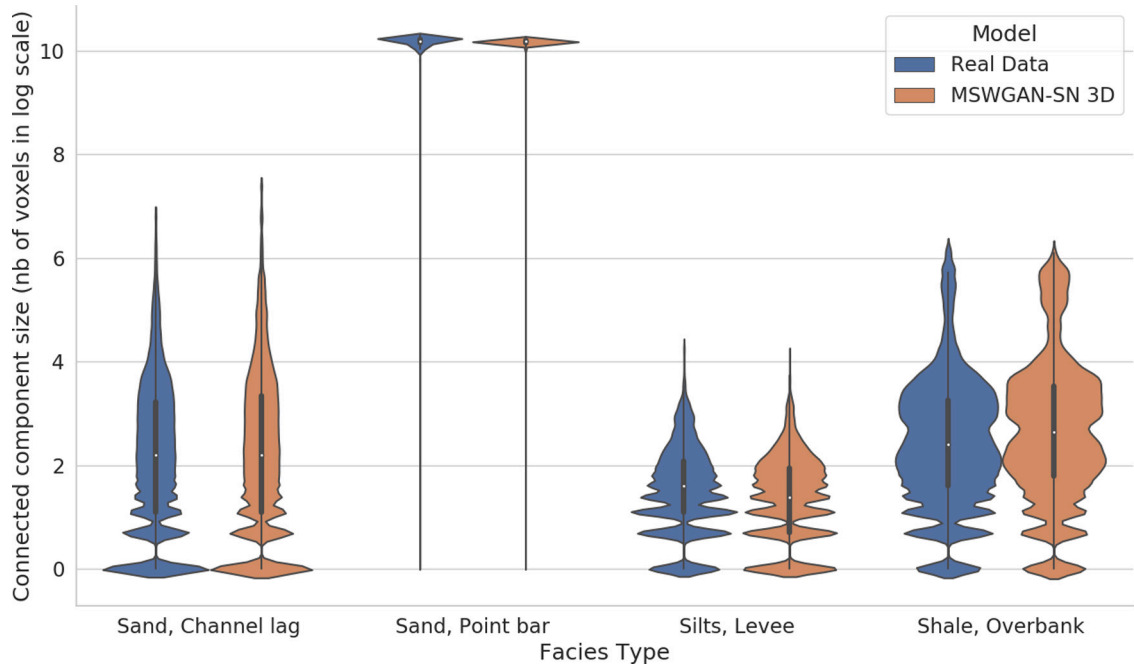


Fig. 9. Distribution of connected components size of each geological facies for the 3D model against training data. To make visualization more relevant, each connected component size is itself weighted by its volume.

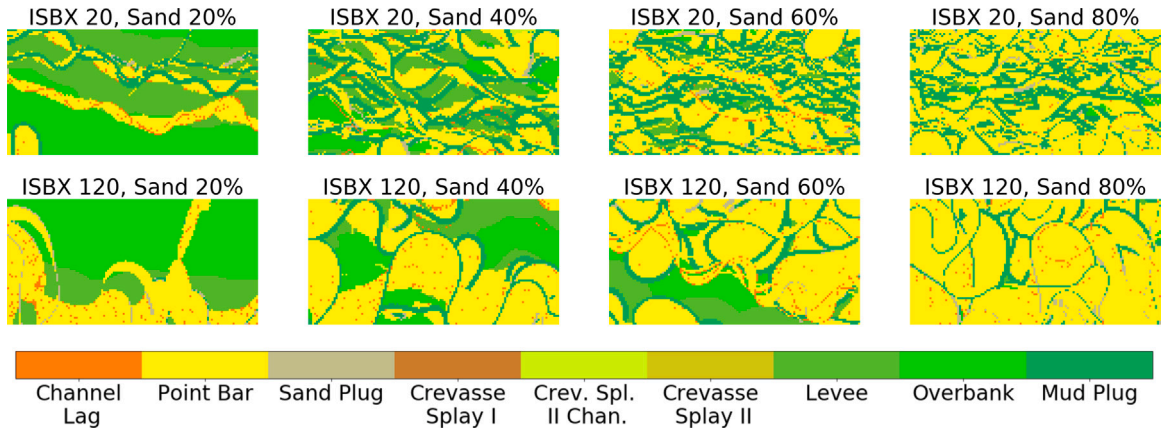


Fig. 10. Realizations generated with Flumy by varying two parameters: the Net to Gross (NG, e.g. the required Sand Proportion) and the Sand Body Extension (ISBX). Flow direction is from left to right. The goal of the parameterization is to capture the relationship between the realizations and the input parameters.

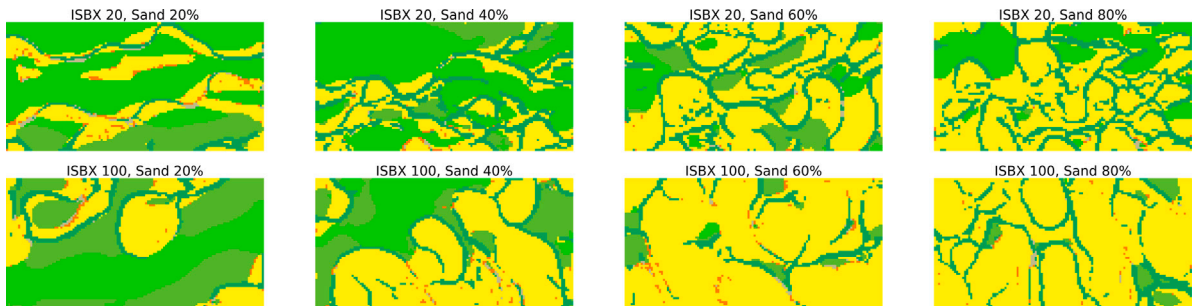


Fig. 11. After training, the CGAN model can generate new realizations, taking into account the parameters it was inputted with.

points, and there are a few pixels with a zero probability. This indicates that the sand channels have no particular trajectories in the rejection-sampling conditional distribution. However, the learned approximation

of the conditional distribution exhibits over-confidence on a small set of sand channel trajectories, which are almost always sampled (in red in the sand facies probability maps). To address this issue, as mentioned

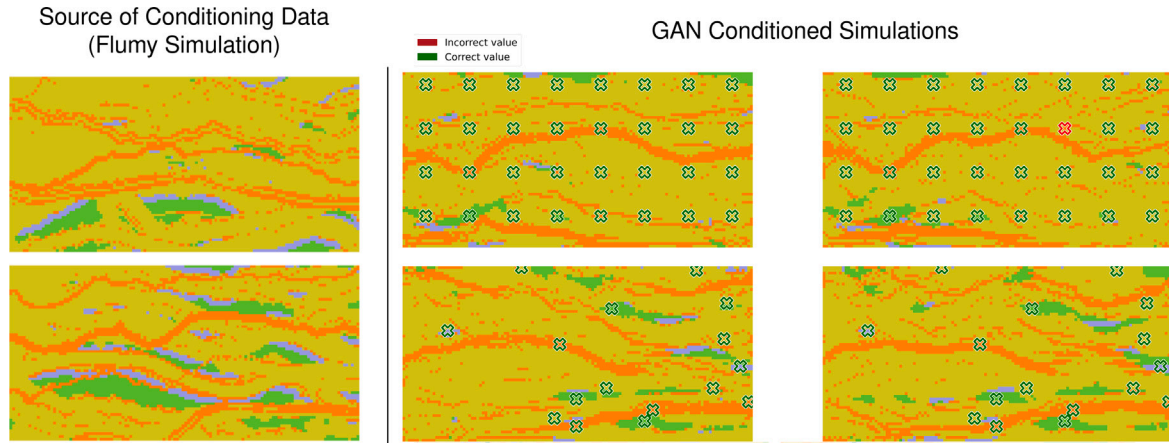


Fig. 12. Example of conditional simulations (boxes, red for an error, green for a correct prediction). Top simulations were conditioned with 32 pixels sampled in a grid-like pattern from a Flumy simulation (leftmost). Bottom simulations were conditioned with 15 pixels sampled in a uniform random pattern from another Flumy simulation (leftmost).

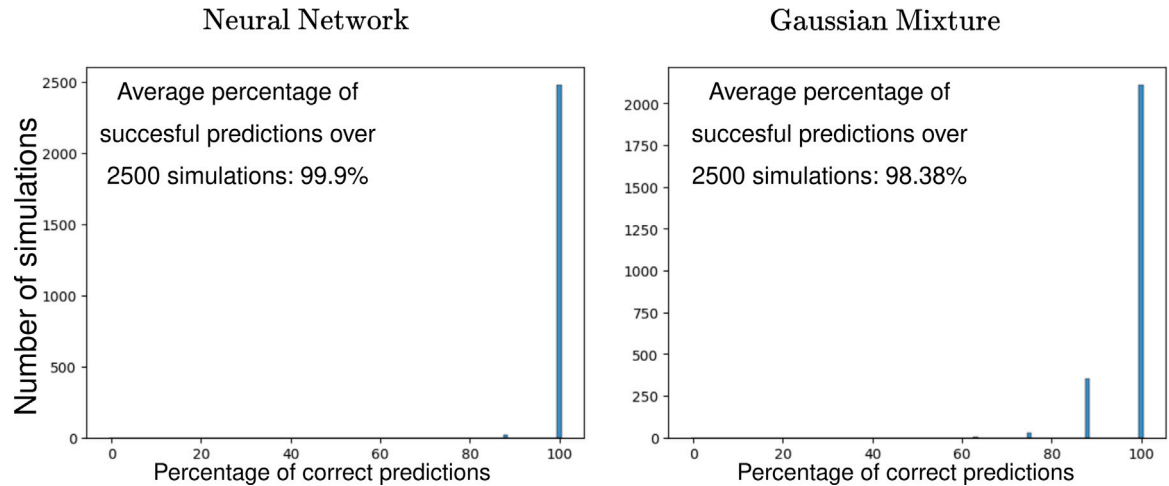


Fig. 13. Comparison of the percentage of success using our two conditioning methods.

in Section 3.2, the neural network model is replaced by a Gaussian Mixture. As can be seen in Fig. 14 and Appendix E, this method generates a probability map closer to that of the rejection method. Indeed, the sand channels do not seem to be constrained to a limited number of trajectories.

5.5.2. 3D

Both conditioning methods are also applicable in 3D, where observation wells correspond to a line of voxels from the top of the volume to the bottom. Two conditional realizations are shown in Fig. 15 along with the original data. In 3D, the differences between the two methods are more blatant. First, the neural network method exhibits, on average, a better match to the conditioning data compared to the mixture method, as shown in Fig. 16. This is a clear advantage of the large expressive power offered by deep learning algorithms. However, the difference in variability is also more pronounced in higher dimension, as seen in Fig. 17. This highlights the advantage of the mixture model to capture a broader range of variations. We have found that 10 is an adequate number of components, enough to exhibit diversity while keeping computation cost as low as possible. Indeed, the main disadvantage of the Gaussian mixture is the training time. Indeed, the covariance matrix of each component is of size $d \times d$, with d the dimension of the

latent space. The algorithm also needs a huge batch size to ensure that every components of the mixture is sufficiently trained at each iteration (e.g., 350 in our experiments).

6. Conclusion

In this paper, we introduced a deep generative model and a method to condition it to observations. Our non-conditional architecture integrates several recent elements from the literature: Wasserstein loss, Multi-Scale generation, non-monotonic 1-Lipschitz bounded activation functions. These elements make the model robust enough to capture the characteristics of datasets generated with a stochastic process-based model, Flumy. Theoretical and empirical analyses support the proposed approach effectiveness in simulating conditional channelized reservoirs. Our experimental results indicate that this approach is successful, in 2D and 3D, as an alternative to Flumy to generate conditional geological facies configurations. The model can be adapted to generate simulations with different parameters, using a CGAN parameterization.

It should be noted that we limited most of our experiments to a specific scenario with a high Net to Gross having 4 facies. Further experimentation needs to be performed to establish the applicability of the conditioning method to more complex Flumy scenarios. However,

Generator simulations accept-reject

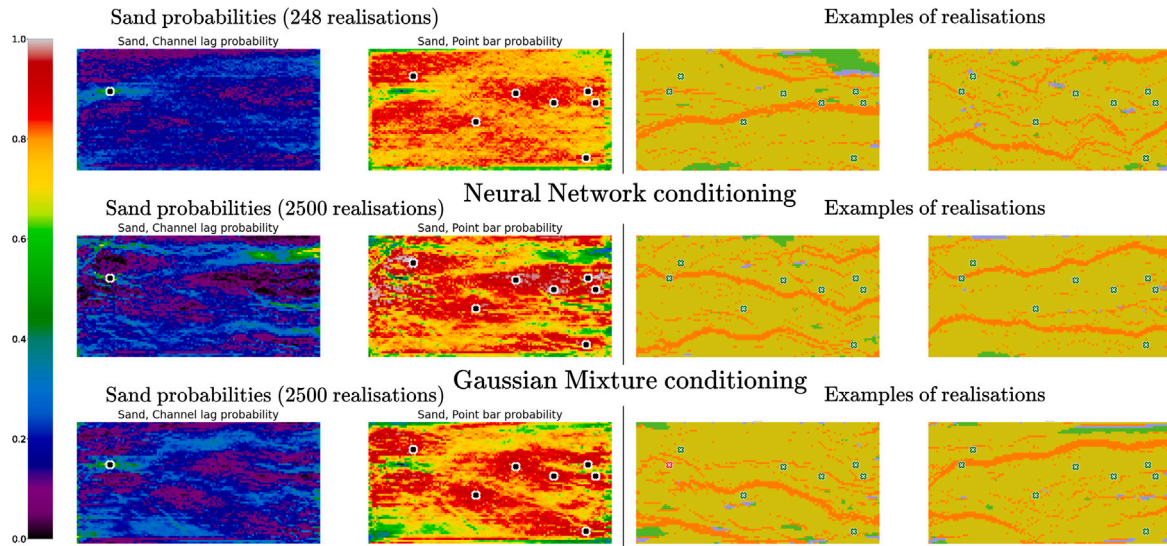


Fig. 14. After the inference model is trained, we can generate any number of realizations. Here we generate 2500 realizations, we compute at each coordinate the probability of occurrence for each facies. We visualize the result (left), as well as sampling two simulations for each methods (right). We compare rejection sampling with the generator, neural network conditioning model and Gaussian mixture conditioning model probabilities.

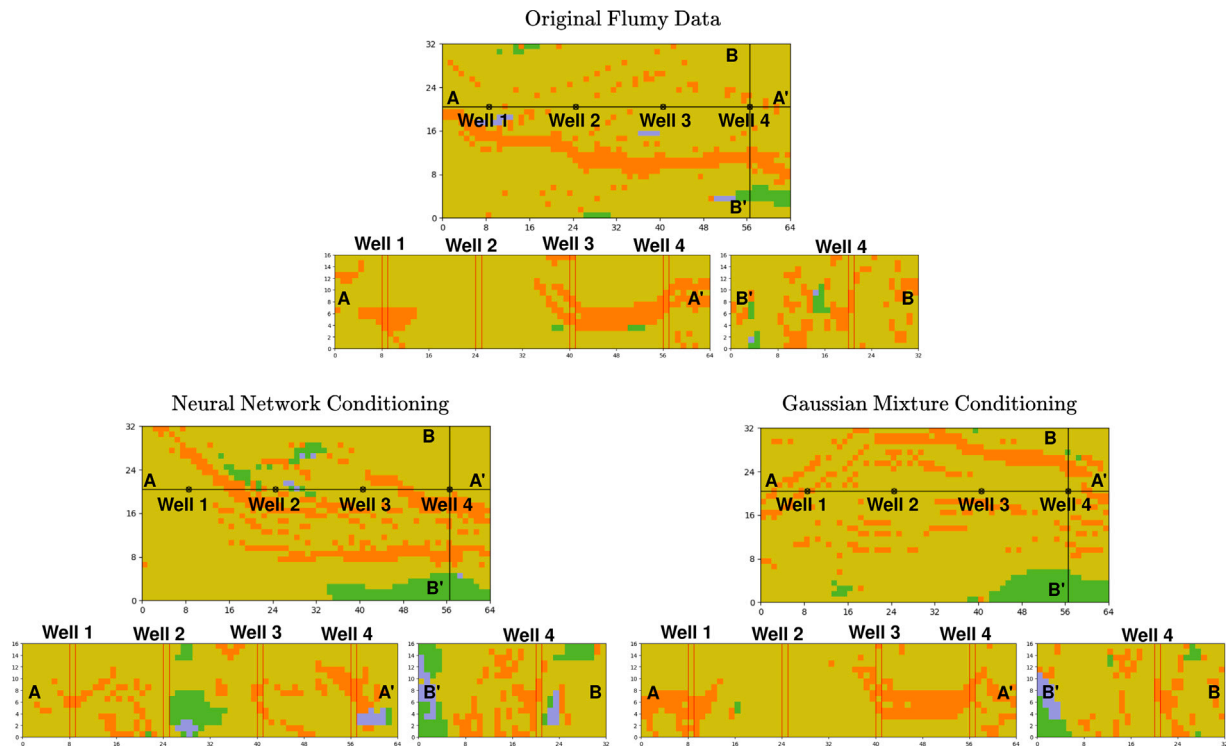


Fig. 15. Conditional simulations with both methods trained in 3D (bottom) compared to the original Flumy volume (top) where the observations were sampled. For each model three slices are shown: The upper one is an horizontal slice of the volume, on the bottom left and bottom right, the vertical sections A-A' and B'-B, respectively.

we believe that the robustness of the non-conditional architecture choices makes the method applicable to other datasets.

We compared our novel Gaussian Mixture-based conditioning method, to the previous Neural Network-based approach. The Gaussian Mixture inference model improves on the Neural Network by better approximating multi-modal distributions. Thus, it shows more variability in the generation of conditioned realizations. This comes at the cost of a higher percentage of inconsistencies with the observations. Nevertheless, the user can easily discard the realizations that do not

honour the conditioning data. Both variational conditioning methods are adaptable such that future research could focus on conditioning to seismic imagery or history matching. Seismic imagery seems the most natural continuation when considering the similarity between seismic proportion maps and low resolution realizations. However, to improve the method, the inference model should be capable of taking advantage of the expressive power and speed of neural networks while being able to have the variability of mixtures. Both approaches could also be

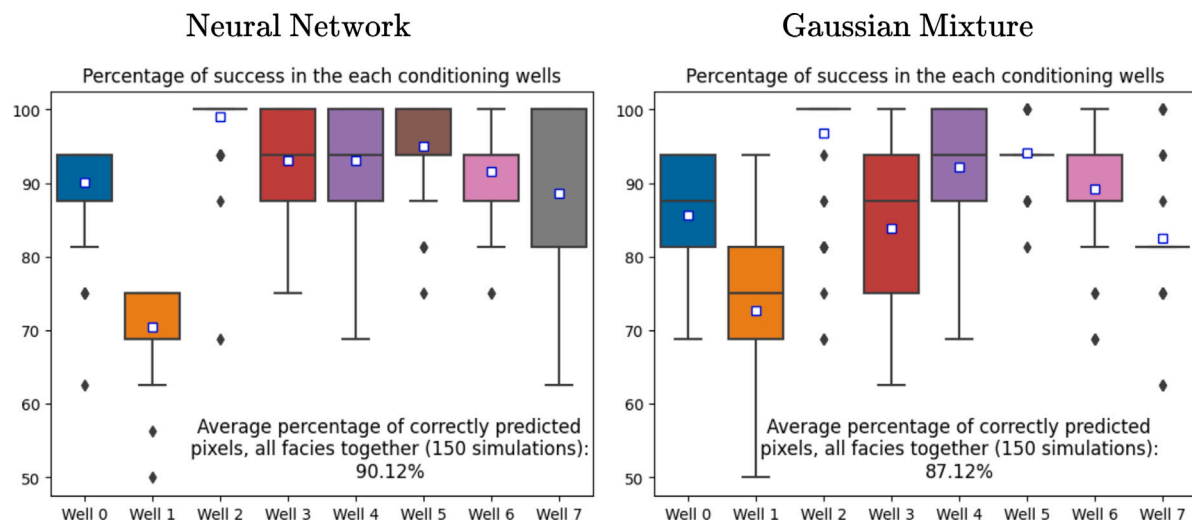


Fig. 16. Boxplot comparing the distribution of the percentage of success, per well, using our two conditioning methods in 3D. Success is defined by whether an observation (a conditioning voxel) has the correct value in the conditioned simulation. The central line of the boxes are the median of the set, blue and white squares are the mean and diamonds are outliers.

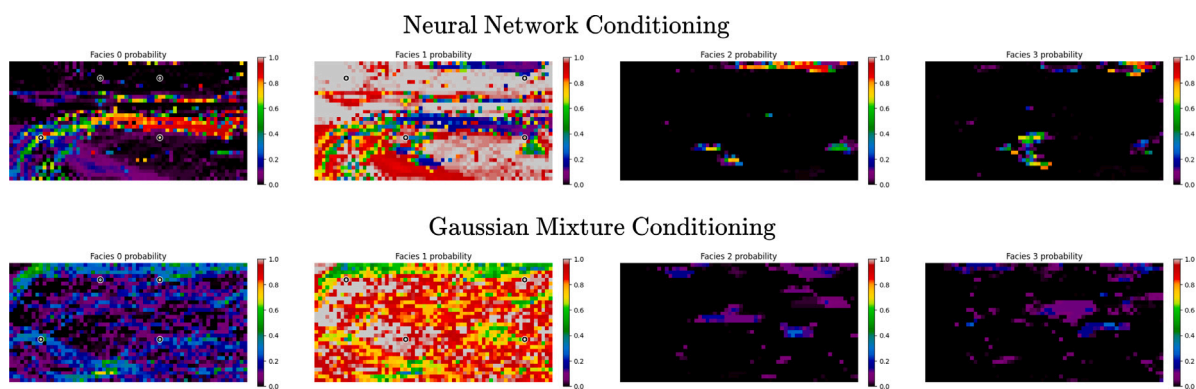


Fig. 17. Identically to the 2D case, we can generate any number of simulations in 3D. Here we generate 150 realizations, we compute at each coordinate the probability of each facies occurring, and we visualize the results of a selected horizontal slice of the 3D volume.

improved by using a prior model (the non-conditional generator) with more expressive power.

CRedit authorship contribution statement

Ferdinand Bhavsar: Writing – original draft, Software, Methodology. **Nicolas Desassis:** Writing – review & editing, Supervision, Methodology, Funding acquisition. **Fabien Ors:** Writing – review & editing, Data curation. **Thomas Romary:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

This research was funded by Institut Carnot M.I.N.E.S, contract n°210000617, and benefited from the support of the Geolearning chair.

This method has been developed within the scope of the Flumy Research Program (JIP). The authors are grateful to ENI partners for support and fruitful discussions.

Code availability section

A stable deep adversarial learning approach for geological facies generation.

The source codes are available for downloading at the link: <https://github.com/Pumafi/stable-gan-for-geol-facies-gen/tree/main>.

Contact: ferdinand.bhavsar@minesparis.psl.eu

Hardware requirements: A GPU with CUDA library installed is highly recommended. 16 GB of System RAM, as well as GPU RAM, is recommended for 3D codes.

Program language: Python

Software required: see requirements.txt

Program size: 57.1 MB (+1.2 GB of data on the git repository)

Appendix A. Final model architecture

This section describes the architecture we recommend to implement our method. It combines a Multi-Scale architecture with Spectral Normalization, and is trained using the Wasserstein loss. This architecture is adapted both to 2D and 3D generation (see Fig. 18).

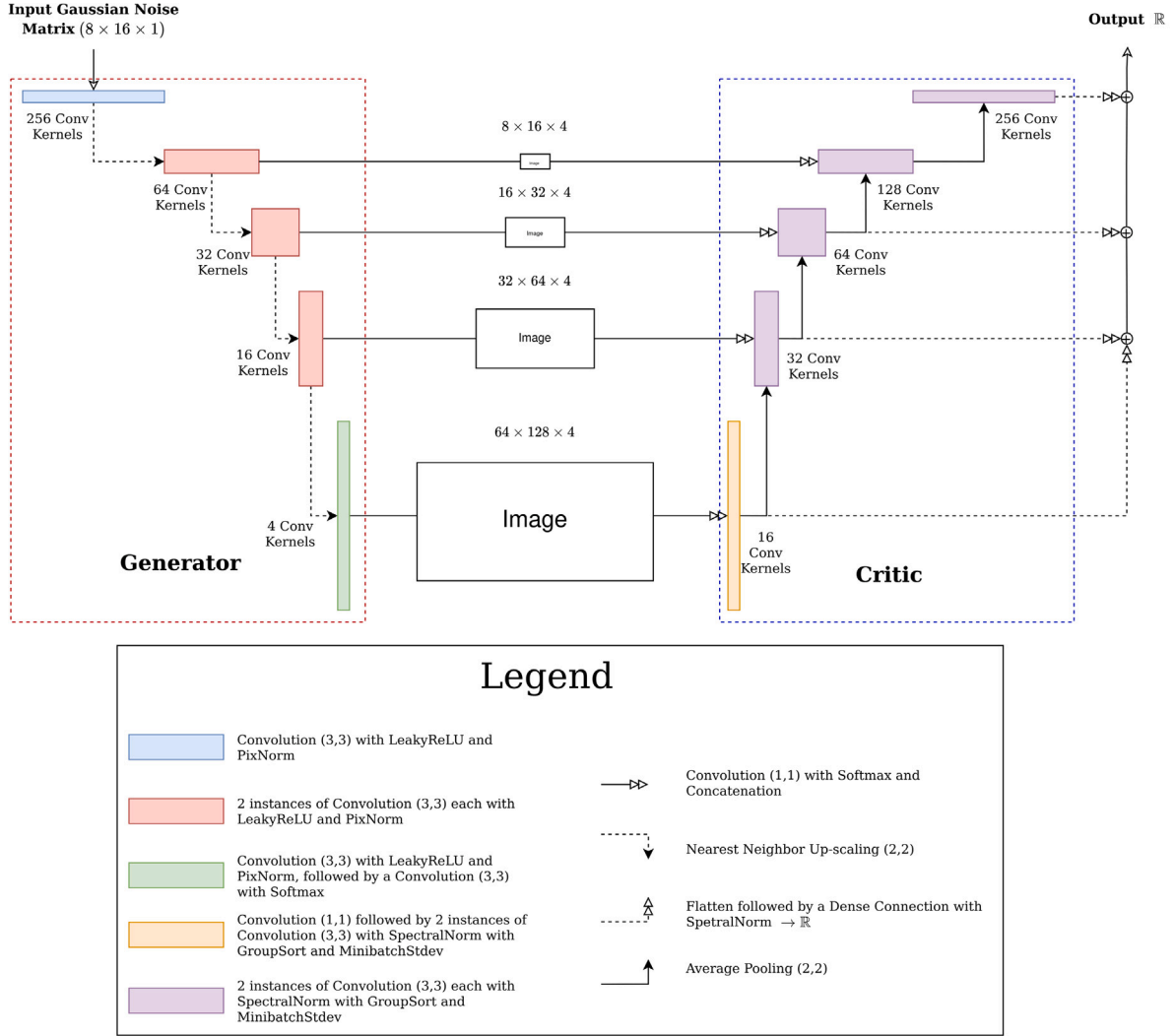


Fig. 18. Diagram of the final MW-GAN-SN-GroupSort model adapted from pre-existing methods and tests.

Table 4

Approximate running time for each models, in seconds per epoch.

Model name	Seconds per epoch	Epochs
2D MSGAN Gaussian Noise	80	100
2D WGAN Gradient Penalty	15	150
2D WGAN SN with GroupSort	140	200
2D MSWGAN SN with GroupSort	90	200
3D MSWGAN SN with NormSwish	230	200

Appendix B. Training time

In this section, we provide the approximate running time for each model, in seconds per epoch. The models were trained using Nvidia T4 GPUs from Google Colaboratory. The architectures presented were not meant to be computationally optimal, and training could be accelerated by improving the models and using recent optimization techniques (see Table 4).

Appendix C. Mathematics

C.1. Wasserstein-GAN and Kantorovich–Rubinstein duality

An efficient method to fix the vanishing gradient problem is to resort to an other loss function. The Wasserstein-1 distance, a dis-

tance measure between two probability distributions, has been proposed (Arjovsky et al., 2017). Unlike the Jensen–Shannon divergence, the derivative of the Wasserstein distance is never constant, such that there is always a useable gradient, making it an ideal candidate to improve the robustness of the networks. It was first defined for optimal transport problems, under the name Earth-Mover distance, to define the minimum effort to move an earth-pile to a hole. There are infinite transport plans to move the earth-pile, but not all are optimal. By changing the earth-pile and hole to probability distributions and the transport plans to joint distributions, we derive a loss function to train a GAN (Arjovsky et al., 2017). Our distributions are the ones defined by the generator model with parameters θ and the training data distribution. They are respectively named p_θ and p_r and are defined on the same probability space M .

A transport plan is noted $\gamma(x, y)$. It is a joint distribution with p_θ and p_r as marginals. The Wasserstein distance is defined as the expectation of the euclidean distance between pairs of random variables following this joint distribution:

$$W(p_r, p_\theta) = \inf_{\gamma \in \Pi(x, y)} \mathbf{E} \|x - y\|_1 \quad (15)$$

We cannot study all joint probability distributions to find the optimum, as the optimization problem is too complex. Instead, we use the simpler form of the Kantorovich–Rubinstein dual (Villani, 2003, 2008).

Theorem C.1.1. Let p_r and p_θ be two probability distributions defined on a probability space M . Let Π be the set of all joint distributions such that p_r and p_θ are the marginal distributions. Let γ be an element of Π and let (x, y) be realizations of γ . The Wasserstein divergence between p_r and p_θ can be expressed in the following dual form:

$$W_1(p_r, p_\theta) = \inf_{\gamma \in \Pi} \mathbf{E}_{(x,y) \sim \gamma} \|x - y\|_1 = \sup_{\substack{f \\ \text{Lip}(f) \leq 1}} \mathbf{E}_{x \sim p_r} f(x) - \mathbf{E}_{y \sim p_\theta} f(y) \quad (16)$$

where the supremum is taken over all functions f that are 1-Lipschitz, which is the property that for any x and y , $f(x) + f(y) \leq \|x - y\|_1$.

We take the Lagrangian associated with the constrained equation, with Lagrange multipliers $f, g : M \rightarrow \mathbb{R}$ (continuous in M) (Villani, 2003, 2008), using the constraints on the transport plan:

$$\begin{aligned} \mathcal{L}(\gamma, f, g) &= \mathbf{E}_{(x,y) \sim \gamma} \|x - y\|_1 + \int_M (P_r(x) - \int_M \gamma(x, y) dy) f(x) dx \\ &\quad + \int_M (P_\theta(y) - \int_M \gamma(x, y) dx) g(y) dy \end{aligned} \quad (17)$$

When $P_r(x) = \int_M \gamma(x, y) dy$ and $P_\theta(y) = \int_M \gamma(x, y) dx$ then we have $\mathcal{L}(\gamma, f, g) = \mathbf{E} \|x - y\|_1$.

However when $P_r(x) \neq \int_M \gamma(x, y) dy$ or $P_\theta(y) \neq \int_M \gamma(x, y) dx$ then $\sup \mathcal{L}(\gamma, f, g) = +\infty$.

Therefore, the optimization problem is equivalent to a min-max problem:

$$W_1(p_r, p_\theta) = \inf_{\gamma \in \Pi} \mathbf{E}_{(x,y) \sim \gamma} \|x - y\|_1 = \inf_{\gamma} \sup_{f, g} \mathcal{L}(\gamma, f, g) \quad (18)$$

We can write the Lagrangian in the following form:

$$\begin{aligned} \mathcal{L}(\gamma, f, g) &= \mathbf{E}_{(x,y) \sim \gamma} \|x - y\|_1 + \int_M (P_r(x) - \int_M \gamma(x, y) dy) f(x) dx \\ &\quad + \int_M (P_\theta(y) - \int_M \gamma(x, y) dx) g(y) dy \\ &= \int_{M \times M} \|x - y\|_1 \gamma(x, y) dx dy + \int_M P_r(x) f(x) dx - \int_{M \times M} f(x) \gamma(x, y) dy dx \\ &\quad + \int_M P_\theta(y) g(y) dy - \int_{M \times M} g(y) \gamma(x, y) dx dy \\ &= \mathbf{E}_{x \sim p_r} f(x) + \mathbf{E}_{y \sim p_\theta} g(y) + \int_{M \times M} (\|x - y\|_1 - f(x) - g(y)) \gamma(x, y) dx dy \end{aligned} \quad (19)$$

The Strong Duality theorem states that if an optimization problem is linear, and the primal and dual form both are realizable, i.e. their constraints are not incompatible, then the optimum of both forms are equal. We can use this theorem because we are in a linear optimization problem and constraints are compatible. Therefore, we rewrite the equation and switch sup and inf.

$$W_1(p_r, p_\theta) = \inf_{\gamma} \sup_{f, g} \mathcal{L}(\gamma, f, g) = \sup_{f, g} \inf_{\gamma} \mathcal{L}(\gamma, f, g) \quad (20)$$

If $\|x - y\|_1 < f^*(x) + g^*(y)$, the inf of the Lagrangian goes to $-\infty$, while if $\|x - y\|_1 \geq f^*(x) + g^*(y)$, then $\inf_{\gamma} \mathcal{L}(\gamma, f, g) = 0$. Therefore we can write:

$$\inf_{\gamma} \int_{M \times M} (\|x - y\|_1 - f(x) - g(y)) \gamma(x, y) dx dy = \begin{cases} 0 & \text{if } \|x - y\|_1 \geq f(x) + g(y) \\ -\infty & \text{else} \end{cases} \quad (21)$$

Working on this Lagrangian allows us to ignore the constraints on γ :

$$\int_{M \times M} (\|x - y\|_1 - f(x) - g(y)) \gamma(x, y) dx dy = \int_{M \times M} F(x, y) \gamma(x, y) dx dy \quad (22)$$

With $F(x, y) = (\|x - y\|_1 - f(x) - g(y))$, $F(x, y) : M \times M \rightarrow \mathbb{R}$, F continuous on its domain as the sum of continuous functions.

If there is a (x, y) such that $F(x, y) < 0$ then, because F is continuous, there exist an interval $I \in M \times M$ where the function takes negative values. This interval is the neighbourhood of (x, y) .

Let us take F such that on interval I the function takes values in $[-\epsilon, -\epsilon/2]$, $\epsilon > 0$, and takes the value 0 on the rest of its domain. We can then write:

$$\begin{aligned} &\int_{M \times M} F(x, y) \gamma(x, y) dx dy \\ &= \int_{(M \times M)/I} F(x, y) \gamma(x, y) dx dy + \int_I F(x, y) \gamma(x, y) dx dy \\ &= \int_I F(x, y) \gamma(x, y) dx dy \\ &\leq -\epsilon/2 \int_I \gamma(x, y) dx dy \end{aligned} \quad (23)$$

By giving $\gamma(x, y)$ an arbitrarily large value on I , we make this integral tend towards $-\infty$. Therefore, when $\|x - y\|_1 < f^*(x) + g^*(y)$, the inf of the Lagrangian tends towards $-\infty$. We then rewrite this part of the equation as a constraint on f and g :

$$W_1(p_r, p_\theta) = \sup_{f, g} \mathbf{E}_{x \sim p_r} f(x) + \mathbf{E}_{y \sim p_\theta} g(y) \quad (24)$$

$$\|x - y\|_1 \geq f(x) + g(y)$$

The problem is now to find the optimal f and the optimal g , i.e. maximize this equation with constraint $\|x - y\|_1 \geq f(x) + g(y)$.

We can show that $g(x) = -f(x)$:

$$\|x - y\|_1 \geq f(x) + g(y) \quad (25)$$

Since we take the sup over all function f and g for which the condition is true, we can determine that:

$$\|x - y\|_1 = f(x) + g(y) \quad (26)$$

This equation is true $\forall x, \forall y$, therefore when $x = y$:

$$f(x) + g(y) = \|x - y\|_1 = 0 \quad (27)$$

Therefore $g(x) = -f(x)$, which gives the following formula:

$$\begin{aligned} W_1(p_r, p_\theta) &= \sup_{f, g} \mathbf{E}_{x \sim p_r} f(x) - \mathbf{E}_{y \sim p_\theta} f(y) \\ &\Leftrightarrow W_1(p_r, p_\theta) = \sup_{\substack{f \\ \text{Lip}(f) \leq 1}} \mathbf{E}_{x \sim p_r} f(x) - \mathbf{E}_{y \sim p_\theta} f(y) \end{aligned} \quad (28)$$

We use this new formulation of the distance to replace the loss function in the previously defined GAN architecture. The 1-Lipschitz function f of the formula is the function defined by our discriminator. Therefore, two major changes are imposed on the architecture:

1. Since f can take any value on $\mathbb{R}$, the last sigmoid activation function present in the original GAN architecture (Goodfellow et al., 2014) is removed.
2. The discriminator is constrained to maintain its Lipschitz bound inferior or equal to 1.

In the original Wasserstein GAN article (Arjovsky et al., 2017), the 1-Lipschitz constraint is enforced through gradient clipping. Gradient clipping simply controls the Lipschitz norm of the model by re-scaling the norm of the gradient to be at most 1. Any gradient norm exceeding this value is re-scaled accordingly. Later studies pointed out that gradient clipping leads to the undesirable behaviour of capacity

underuse, leading to the critic ending up learning extremely simple functions (Gulrajani et al., 2017). These over-simplified functions are a problem, as they might fail to represent the Wasserstein loss correctly. As such we leverage properties on the norm of the gradient (see Appendices C.2 and C.3) and use Spectral Normalization (see Section 2).

C.2. Lipschitz norm of Kantorovich–Rubinstein dual

Theorem C.2.1. *Given γ^* the optimal transport plan minimizing:*

$\inf_{\gamma \in \Pi} \mathbb{E}_{(x,y) \sim \gamma} \|x - y\|_1$. If f^ , the solution of the Kantorovich–Rubinstein dual (also called optimal critic) is differentiable, then we have the following equality:*

$$P[f^*(x) - f^*(y) = \|x - y\|_1] = 1 \quad (29)$$

Thanks to Theorem C.1.1 we can write:

$$\sup_{f \in \Pi} \mathbb{E}_{x \sim P_r} f(x) - \mathbb{E}_{y \sim P_\theta} f(y) = \inf_{\gamma \in \Pi} \mathbb{E}_{(x,y) \sim \gamma} \|x - y\|_1 \quad (30)$$

Because of the Strong Duality theorem we know both optimal values are equal, and we can write:

$$\begin{aligned} \mathbb{E}_{(x,y) \sim \gamma^*} [f^*(x) - f^*(y)] &= \mathbb{E}_{(x,y) \sim \gamma^*} \|x - y\|_1 \\ \Leftrightarrow \mathbb{E}_{(x,y) \sim \gamma^*} [f^*(x) - f^*(y)] - \mathbb{E}_{(x,y) \sim \gamma^*} \|x - y\|_1 &= 0 \\ \Leftrightarrow \mathbb{E}_{(x,y) \sim \gamma^*} [f^*(x) - f^*(y) - \|x - y\|_1] &= 0 \end{aligned} \quad (31)$$

The expectation is zero, therefore the probability distribution is centred. But since the function is 1-Lipschitz continuous, we can write $f^*(y) - f^*(x) \leq \|y - x\|_1$. Both properties are only possible if all the mass of the probability distribution is at 0.

C.3. Optimal critic gradient

Theorem C.3.1. *If $f^* : \mathbb{R}^n \rightarrow \mathbb{R}$, $n \in \mathbb{N}$, is the 1-Lipschitz optimal critic that maximizes the Kantorovich–Rubinstein dual, then $\|\nabla f^*(x)\|_1 = 1$ almost everywhere.*

Using the Frechet derivative generalization to vector-valued functions we write the derivative of f :

$$\begin{aligned} \lim_{\|x-y\|_1 \rightarrow 0} \frac{f^*(x) - f^*(y) - \langle \nabla f^*(x), y - x \rangle}{\|x - y\|_1} &= 0 \\ \Leftrightarrow \lim_{\|x-y\|_1 \rightarrow 0} \frac{f^*(x) - f^*(y)}{\|x - y\|_1} &= \lim_{\|x-y\|_1 \rightarrow 0} \frac{\langle \nabla f^*(x), y - x \rangle}{\|x - y\|_1} \end{aligned} \quad (32)$$

Using the Cauchy–Schwarz inequality we deduce a lower bound of this derivative:

$$\begin{aligned} \lim_{\|x-y\|_1 \rightarrow 0} \frac{f^*(x) - f^*(y)}{\|x - y\|_1} &= \lim_{\|x-y\|_1 \rightarrow 0} \frac{\langle \nabla f^*(x), y - x \rangle}{\|x - y\|_1} \leq \frac{\|\nabla f^*(x)\|_1 \cdot \|y - x\|_1}{\|y - x\|_1} = \|\nabla f^*(x)\|_1 \end{aligned} \quad (33)$$

Thanks to Theorem C.2.1 we know that $P\left[\frac{f^*(x) - f^*(y)}{\|x - y\|_1} = 1\right] = 1$. Consequently $P[\|\nabla f^*(x)\|_1 \geq 1] = 1$, i.e. 1 is a lower bound of the gradient almost everywhere. Since the function is 1-Lipschitz, the upper bound of the gradient is also 1. The upper and lower bound of the gradient are equal almost everywhere, therefore we proved that $\|\nabla f^*(x)\|_1 = 1$ almost everywhere.

Arjovsky and Bottou (2017) uses gradient penalty to enforce this theorem locally on the critic. Enforcing it on the entirety of its domain of definition require to make every component of the critic 1-Lipschitz continuous. Linear and Convolutional layers can be constrained to estimating 1-Lipschitz function by using Spectral Normalization (Miyato et al., 2018).

C.4. Kozachenko–Leonenko entropy estimator

Given a random variable $\tilde{z}$ with unknown probability distribution q_θ defined by our inference network I , which we can sample. If we have M realizations $\tilde{z}_{i0 \leq i \leq M}$, then for any $\tilde{z}$ we can compute the k nearest neighbours in the set of realization $\tilde{z}_i$. We note the distance from $\tilde{z}$ to the k th neighbour $d_{knn}(\tilde{z})$. $B(\tilde{z}, d_{knn}(\tilde{z}))$ is smallest ball with $\tilde{z}$ as centre containing its k nearest neighbours. Its radius is $d_{knn}(\tilde{z})$.

If we take the assumption that q is continuous and $q(\tilde{z})_{q\tilde{z}} \approx q(y)_{y \in B(\tilde{z}, d_{knn}(\tilde{z}))}$, then we can estimate the empirical Shannon entropy of q_θ using our M realizations $\tilde{z}_{i0 \leq i \leq M}$, thanks to the Kozachenko–Leonenko estimator (L.F. Kozachenko, 1987):

$$\hat{H}(q) \approx \log(M) - \log(k) + \log(V_{B(1)}) + \frac{d}{M} \sum_{i=0}^M \log(d_{knn}(\tilde{z}_i)) \quad (34)$$

Intuitively, what this formula means is that the closer is the k th nearest neighbour from $\tilde{z}$, the lower is the entropy of $q_\theta(\tilde{z})$.

Only $\frac{d}{M} \sum_{i=0}^M \log(d_{knn}(\tilde{z}_i))$ is optimized in this equation. Therefore, we can replace the entropy in our loss function:

$$\begin{aligned} \operatorname{argmin}_{\theta} D_{KL}(q_\theta(z|x^*) \parallel p(z|x^*)) \\ \approx \operatorname{argmin}_{\theta} -\frac{d}{M} \sum_{i=0}^M \log(d_{knn}(\tilde{z}_i)) + \mathbb{E}_{z \sim q_\theta} \left[-\sum_{i=1}^N \log p(x_i^*|z) + \frac{1}{2} \|z\|^2 \right] \end{aligned} \quad (35)$$

Appendix D. Transition probabilities

In this section, we further analyse the MW-GAN-SN model by studying the transition probabilities from one facies to another as a function of the distance (see Fig. 19).

Appendix E. Kolmogorov–Smirnov test for conditioning

We extend the analysis of the two conditioning methods, neural network and Gaussian mixture, using the Kolmogorov–Smirnov test. We compare the sand facies probability distributions between our conditional models and accept–rejects samples using the Kolmogorov–Smirnov two-sample test at a significance level of 0.05. A p -value higher than this threshold indicates that the null hypothesis holds and that the two samples can be considered to be from similar distributions (see Fig. 20).

Appendix F. Training data generation process

Training datasets have been generated using FLUMY v6.004, Process-based channelized reservoir models, Copyright MINES PARIS PSL/ARMINES, Free download from <https://flumy.minesparis.psl.eu>. The non-conditional Flumy reference simulation used in this study aims at reproducing channelized reservoirs made by a small aggrading turbidite inside a canyon having a length of 1280 m, a width of 640 m and a height of 640 m. The turbidite channel has a maximum thickness of 10 m, a constant width of 350 m and a large wavelength of 4000 m. The erodibility coefficient is reduced to $1e-08$ in order to slow down lateral migration (low sinuosity). Aggradation events occur in average every 140 iterations with an intensity of 1 m thickness, a small horizontal overbank extension of 928 m and a small levee width of 140 m. Local avulsions are disabled and regional avulsions period is set to 110 iterations. This parameter set results in a simulation having a very high sand proportion with elongated channel lag footprints along the flow direction and some scattered shale lenses spread into the facies block that can constitute flow barriers. Flumy reference parameters are summarized in Table 5. All parameters that are not cited in this table are kept to their default value.

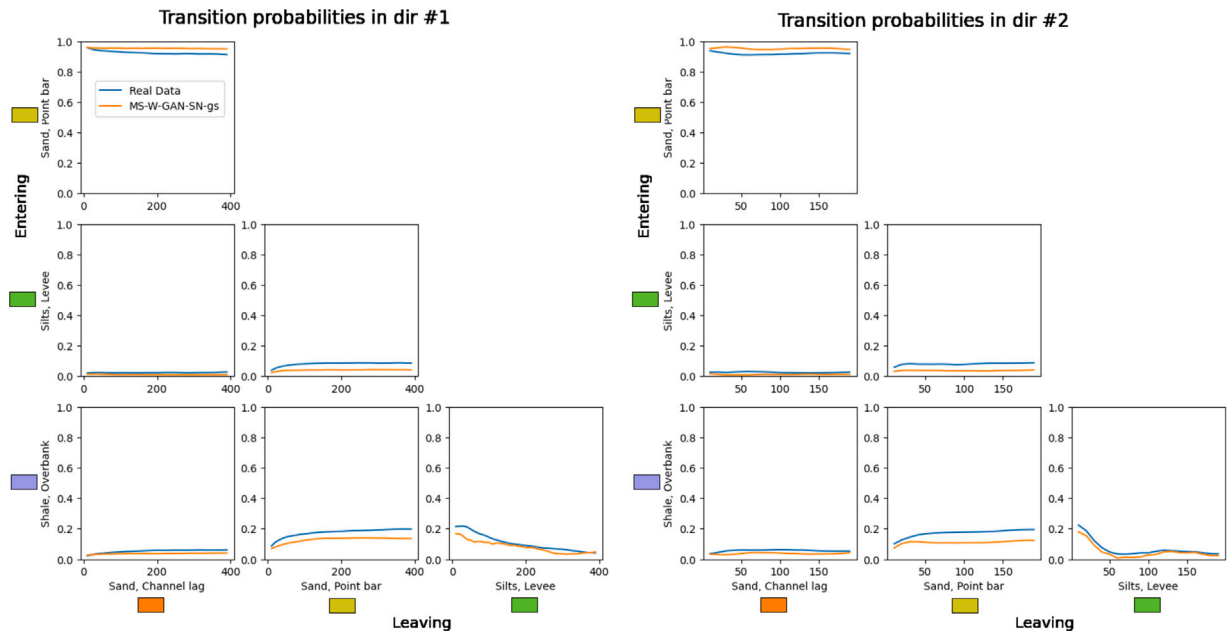
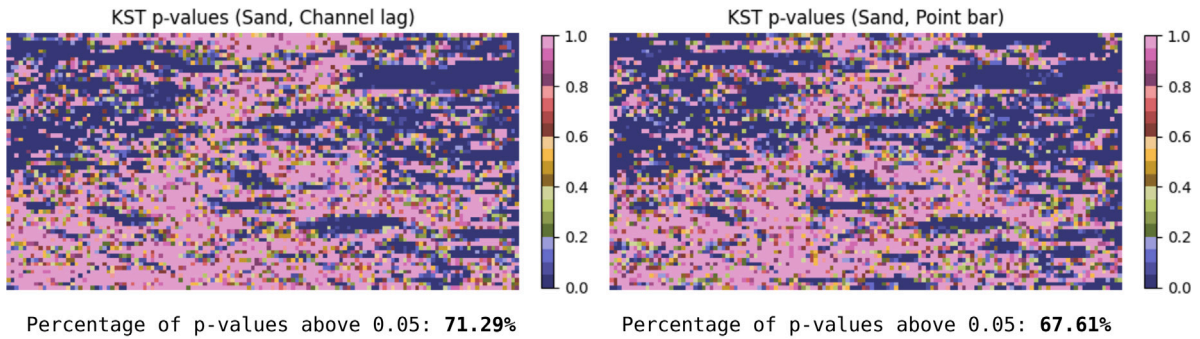


Fig. 19. Transition Probabilities for the 2D MW-GAN-SN with GroupSort architecture. Direction 1 is the width direction, while the direction 2 is the height direction.

Neural Network conditioning



Gaussian Mixture conditioning

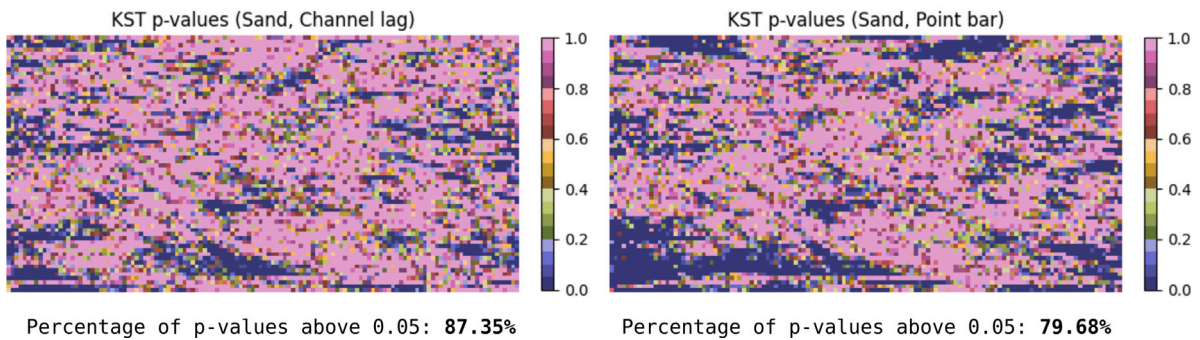


Fig. 20. Comparison of Kolmogorov-Smirnov tests for neural network and Gaussian mixture.

Table 5

Flumy parameters used for our Flumy reference simulation.

Parameter	Value
Grid	
Number of Nodes	128 × 64
Mesh	50 × 50 m
Block Thickness	640 m
Vertical Discretization	0.5 m
Channel	
Maximum Depth	10 m
Width	350 m
Wavelength	4000 m
Erodibility Coefficient	1e-08 m/s
Overbank	
Period (Poisson)	140it
Intensity (Normal)	140it
Extension	928 m
Avulsions	
Local Period	Never
Regional Period (Poisson)	110it

References

- Abdellatif, A., Elsheikh, A.H., Busby, D., Berthet, P., 2023. Generation of non-stationary stochastic fields using generative adversarial networks. URL <http://arxiv.org/abs/2205.05469>, cs.
- Anil, C., Lucas, J., Grosse, R.B., 2018. Sorting out Lipschitz function approximation. CoRR, arXiv:1811.05381, URL <http://arxiv.org/abs/1811.05381>.
- Arjovsky, M., Bottou, L., 2017. Towards principled methods for training generative adversarial networks. In: Conference on Computer Vision and Pattern Recognition.
- Arjovsky, M., Chintala, S., Bottou, L., 2017. Wasserstein generative adversarial networks. In: International Conference on Machine Learning. PMLR, pp. 214–223.
- Armstrong, M., Galli, A., Loch, G.L., Geoffroy, F., Eschard, R., 2001. Pluri-Gaussian Simulations. Springer, Berlin.
- Brock, A., Donahue, J., Simonyan, K., 2019. Large scale GAN training for high fidelity natural image synthesis. <http://dx.doi.org/10.48550/arXiv.1809.11096>, URL <http://arxiv.org/abs/1809.11096>, arXiv:1809.11096[cs,stat].
- Bubnova, A., 2018. On the Conditioning of Process-Based Channelized Meandering Reservoir Models on Well Data (Ph.D. thesis). (2018PSLEM055), Université Paris sciences et lettres, URL <https://pastel.archives-ouvertes.fr/tel-02173727>.
- Caers, J., 2005. Petroleum Geostatistics. Society of Petroleum Engineers Richardson.
- Canchumuni, S.W., Emerick, A.A., Pacheco, M.A.C., 2019. Towards a robust parameterization for conditioning facies models using deep variational autoencoders and ensemble smoother. Comput. Geosci. 128, 87–102. <http://dx.doi.org/10.1016/j.cageo.2019.04.006>, URL <https://www.sciencedirect.com/science/article/pii/S0098300419300378>.
- Chan, S., Elsheikh, A.H., 2017. Parametrization and generation of geological models with generative adversarial networks. arXiv:1708.01810.
- Chan, S., Elsheikh, A., 2019. Parametric generation of conditional geological realizations using generative neural networks. Comput. Geosci. 23, <http://dx.doi.org/10.1007/s10596-019-09850-7>.
- Chen, M., Wu, S., Bedle, H., Xie, P., Zhang, J., Wang, Y., 2022. Modeling of subsurface sedimentary facies using self-attention generative adversarial networks (SAGANs). J. Pet. Sci. Eng. 214, 110470. <http://dx.doi.org/10.1016/j.petrol.2022.110470>, URL <https://linkinghub.elsevier.com/retrieve/pii/S0920410522003540>.
- Chilès, J.P., Delfiner, P., 2012. Geostatistics, Modeling Spatial Uncertainty, second ed. John Wiley & Sons, New-York.
- Chiu, S.N., Stoyan, D., Kendall, W.S., Mecke, J., 2013. Stochastic Geometry and Its Applications. John Wiley & Sons.
- Dubrule, O., 1989. A review of stochastic models for petroleum reservoirs. Geostatistics 493–506.
- Dupont, E., Zhang, T., Tilke, P., Liang, L., Bailey, W., 2018. Generating realistic geology conditioned on physical measurements with generative adversarial networks. arXiv:1802.03065.
- Flumy-Userguide, 2022. FLUMY: Process-based channelized reservoir models. MINES Paris - PSL / ARMINES, Fontainebleau, France, Free download from: <https://flumy.minesparis.psl.eu>.
- Focaccia, S., Tinti, F., Monti, F., Amidei, S., Bruno, R., 2016. Shallow geothermal energy for industrial applications: A case study. Sustain. Energy Technol. Assess. 16, 93–105. <http://dx.doi.org/10.1016/j.seta.2016.05.003>, URL <https://www.sciencedirect.com/science/article/pii/S221313881630025X>.
- Fossum, K., Alyaev, S., Elsheikh, A.H., 2024. Ensemble history-matching workflow using interpretable SPADE-fan geomodel. First Break 42 (2), 57–63. <http://dx.doi.org/10.3997/1365-2397.fb2024014>, URL <https://www.earthdoc.org/content/journals/10.3997/1365-2397.fb2024014>.
- Fossum, K., Alyaev, S., Tveranger, J., Elsheikh, A.H., 2022. Verification of a real-time ensemble-based method for updating earth model based on GAN. J. Comput. Sci. 65, 101876. <http://dx.doi.org/10.1016/j.jocs.2022.101876>, URL <https://www.sciencedirect.com/science/article/pii/S1877750322002356>.
- Furuya, T., Kusumoto, H., Taniguchi, K., Kanno, N., Suetake, K., 2022. Variational inference with Gaussian mixture by entropy approximation.
- Geman, S., Geman, D., 1984. Stochastic relaxation, gibbs distributions, and the Bayesian restoration of images. IEEE Trans. Pattern Anal. Mach. Intell. (6), 721–741.
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., Bengio, Y., 2014. Generative adversarial nets. In: Advances in Neural Information Processing Systems, pp. 2672–2680.
- Grimaud, J.L., Ors, F., Lemay, M., Cojan, I., Rivoirard, J., 2022. Preservation and completeness of fluvial meandering deposits influenced by channel motions and overbank sedimentation. J. Geophys. Res.: Earth Surf. 127 (5), <http://dx.doi.org/10.1029/2021JF006435>, e2021JF006435, arXiv:<https://agupubs.onlinelibrary.wiley.com/doi/pdf/10.1029/2021JF006435>, URL <https://agupubs.onlinelibrary.wiley.com/doi/abs/10.1029/2021JF006435>, e2021JF006435.
- gstlearn, 2023. GStlearn: Geostatistics & Machine Learning Toolbox. MINES Paris - PSL, Fontainebleau, France, Free download from: <https://gstlearn.org>.
- Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., Courville, A., 2017. Improved training of wasserstein gans. arXiv preprint arXiv:1704.00028.
- Hochreiter, S., 1991. Untersuchungen zu dynamischen neuronalen netzen.
- Jacod, J., Joathon, P., 1971. Use of random-genetic models in the study of sedimentary process. Math. Geol. 3 (3), 219–233.
- Jetchev, N., Bergmann, U., Vollgraf, R., 2016. Texture synthesis with spatial generative adversarial networks. <http://dx.doi.org/10.48550/ARXIV.1611.08207>, URL <https://arxiv.org/abs/1611.08207>.
- Jeulin, D., 2021. Morphological Models of Random Structures. Springer.
- Karnewar, A., Wang, O., Iyengar, R.S., 2019. MSG-GAN: multi-scale gradient GAN for stable image synthesis. CoRR, arXiv:1903.06048, URL <http://arxiv.org/abs/1903.06048>.
- Karras, T., Aila, T., Laine, S., Lehtinen, J., 2017. Progressive growing of GANs for improved quality, stability, and variation. CoRR, arXiv:1710.10196, URL <http://arxiv.org/abs/1710.10196>.
- Karras, T., Laine, S., Aila, T., 2019. A style-based generator architecture for generative adversarial networks. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 4401–4410.
- Kidger, P., Lyons, T.J., 2019. Universal approximation with deep narrow networks. CoRR, arXiv:1905.08539, URL <http://arxiv.org/abs/1905.08539>.
- Kitanidis, P.K., 1997. Introduction to Geostatistics: Applications in Hydrogeology. Cambridge University Press.
- Klambauer, G., Unterthiner, T., Mayr, A., Hochreiter, S., 2017. Self-normalizing neural networks. CoRR, arXiv:1706.02515, URL <http://arxiv.org/abs/1706.02515>.
- Laloy, E., Héroult, R., Jacques, D., Linde, N., 2018. Training-image based geostatistical inversion using a spatial generative adversarial neural network. Water Resour. Res. 54 (1), 381–406. <http://dx.doi.org/10.1002/2017WR022148>, arXiv:<https://agupubs.onlinelibrary.wiley.com/doi/pdf/10.1002/2017WR022148>, URL <https://agupubs.onlinelibrary.wiley.com/doi/abs/10.1002/2017WR022148>.
- Laloy, E., Héroult, R., Lee, J., Jacques, D., Linde, N., 2017. Inversion using a new low-dimensional representation of complex binary geological media based on a deep neural network. Adv. Water Resour. 110, 387–405. <http://dx.doi.org/10.1016/j.advwatres.2017.09.029>, URL <https://hal.archives-ouvertes.fr/hal-02094960>.
- Langanay, J., Romary, T., Freulon, X., Langlais, V., Petit, G., Lagneau, V., 2021. Uncertainty quantification for uranium production in mining exploitation by in situ recovery. Comput. Geosci. 25 (3), 831–850.
- Lantuéjoul, C., 2002. Geostatistical Simulation. Springer, Berlin.
- L.F. Kozachenko, N.L., 1987. Statistical estimate for the entropy of a random vector. Probl. Pereda. Inf. 23, 9–16.
- Linde, N., Renard, P., Mukerji, T., Caers, J., 2015. Geological realism in hydrogeological and geophysical inverse modeling: A review. Adv. Water Resour. 86, 86–101. <http://dx.doi.org/10.1016/j.advwatres.2015.09.019>, URL <https://www.sciencedirect.com/science/article/pii/S0309170815002262>.
- Liu, X., Chen, X., Cheng, J., Zhou, L., Chen, L., Li, C., Zu, S., 2023. Simulation of complex geological architectures based on multistage generative adversarial networks integrating with attention mechanism and spectral normalization. IEEE Trans. Geosci. Remote Sens. 61, 1–15. <http://dx.doi.org/10.1109/TGRS.2023.3294493>, URL <https://ieeexplore.ieee.org/document/10179966/>.
- Liu, X., Cheng, J., Cai, Y., Mo, Q., Li, C., Zu, S., 2022. Stochastic simulation of facies using deep convolutional generative adversarial network and image quilting. <http://dx.doi.org/10.2139/ssrn.4150478>, URL <https://papers.ssrn.com/abstract=4150478>.
- Long, J., Shelhamer, E., Darrell, T., 2015. Fully convolutional networks for semantic segmentation. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. pp. 3431–3440.
- Lopez, S., 2003a. Channelized reservoir modeling: A stochastic process-based approach.
- Lopez, S., 2003b. Channelized reservoir modeling: A stochastic process-based approach.
- Mariethoz, G., Caers, J., 2014. Multiple-Point Geostatistics: Stochastic Modeling with Training Images. John Wiley & Sons.
- Matheron, G., 1966. Éléments pour une théorie des milieux poreux. Masson Paris.

- Matheron, G., Beucher, H., de Fouquet, C., Galli, A., Guerillot, D., Ravenne, C., 1987. Conditional simulation of the geometry of fluvio-deltaic reservoirs. In: *Spe Annual Technical Conference and Exhibition*. OnePetro.
- Mirza, M., Osindero, S., 2014. Conditional generative adversarial nets. *arXiv preprint arXiv:1411.1784*.
- Miyato, T., Kataoka, T., Koyama, M., Yoshida, Y., 2018. Spectral normalization for generative adversarial networks. <http://dx.doi.org/10.48550/ARXIV.1802.05957>, URL <https://arxiv.org/abs/1802.05957>.
- Mosser, L., Dubrule, O., Blunt, M.J., 2017. Reconstruction of three-dimensional porous media using generative adversarial neural networks. *Phys. Rev. E* 96, 043309. <http://dx.doi.org/10.1103/PhysRevE.96.043309>, URL <https://link.aps.org/doi/10.1103/PhysRevE.96.043309>.
- Narayanan, H., Mitter, S., 2010. Sample complexity of testing the manifold hypothesis. In: Lafferty, J., Williams, C., Shawe-Taylor, J., Zemel, R., Culotta, A. (Eds.), In: *Advances in Neural Information Processing Systems*, vol. 23, Curran Associates, Inc., URL <https://proceedings.neurips.cc/paper/2010/file/8a1e808b55fde9455cb3d8857ed88389-Paper.pdf>.
- Pinkus, A., 1999. Approximation theory of the MLP model in neural networks. *Acta Numer.* 8, 143–195. <http://dx.doi.org/10.1017/S0962492900002919>.
- Ramachandran, P., Zoph, B., Le, Q.V., 2018. Searching for activation functions. URL <https://openreview.net/forum?id=SkBYyZRZ>.
- Song, S., Mukerji, T., Hou, J., 2021. Geological facies modeling based on progressive growing of generative adversarial networks (GANs). *Comput. Geosci.* 25, <http://dx.doi.org/10.1007/s10596-021-10059-w>.
- Song, S., Mukerji, T., Hou, J., Zhang, D., Lyu, X., 2022. GANSim-3D for conditional geomodeling: Theory and field application. *Water Resour. Res.* 58 (7), <http://dx.doi.org/10.1029/2021WR031865>, e2021WR031865, URL <https://onlinelibrary.wiley.com/doi/abs/10.1029/2021WR031865>, eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1029/2021WR031865>.
- Strebel, S., 2002. Conditional simulation of complex geological structures using multiple-point geostatistics. *Math. Geol.* 34, 1–22.
- Sun, C., Demyanov, V., Arnold, D., 2023a. Geological realism in fluvial facies modelling with GAN under variable depositional conditions. *Comput. Geosci.* 27 (2), 203–221. <http://dx.doi.org/10.1007/s10596-023-10190-w>.
- Sun, C., Demyanov, V., Arnold, D., 2023b. Geological realism in fluvial facies modelling with GAN under variable depositional conditions. *Comput. Geosci.* <http://dx.doi.org/10.1007/s10596-023-10190-w>.
- Troncoso, A., 2022. Conditional Simulations of Reservoir Models Using Sequential Monte-Carlo Methods (Ph.D. thesis). (2022UPSLM055), Université Paris sciences et lettres, URL <https://pastel.hal.science/tel-04077499>.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I., 2017. Attention is all you need. In: Guyon, I., Luxburg, U.V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., Garnett, R. (Eds.), In: *Advances in Neural Information Processing Systems*, vol. 30, Curran Associates, Inc., URL <https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf>.
- Villani, C., 2003. *Topics in Optimal Transportation*. American Mathematical Soc., Google-Books-ID: idyFawAAQBAJ.
- Villani, C., 2008. *Optimal Transport: Old and New*. Springer Science & Business Media, Google-Books-ID: hV8o5R7_5tkC.
- Wenliang, L.K., Kanagawa, H., 2020. Blindness of score-based methods to isolated components and mixing proportions. *arXiv preprint arXiv:2008.10087*.
- Zhang, W., Li, H., Yongqin, L., Liu, H., Chen, Y., Ding, X., 2021. Application of deep learning algorithms in geotechnical engineering: A short critical review. *Artif. Intell. Rev.* 54, 1–41. <http://dx.doi.org/10.1007/s10462-021-09967-1>.
- Zhang, T., Yang, Z., Li, D., 2022. Stochastic simulation of deltas based on a concurrent multi-stage VAE-GAN model. *J. Hydrol.* 607, 127493. <http://dx.doi.org/10.1016/j.jhydrol.2022.127493>, URL <https://linkinghub.elsevier.com/retrieve/pii/S0022169422000683>.
- Zhong, Z., Carr, T.R., 2019. Geostatistical 3D geological model construction to estimate the capacity of commercial scale injection and storage of CO2 in jacksonburg-stringtown oil field, west virginia, USA. *Int. J. Greenh. Gas Control* 80, 61–75. <http://dx.doi.org/10.1016/j.ijggc.2018.10.011>, URL <https://www.sciencedirect.com/science/article/pii/S1750583618301567>.
- Zhu, M., Min, W., Wang, Q., Zou, S., Chen, X., 2021. PFLU and FPFLU: Two novel non-monotonic activation functions in convolutional neural networks. *Neurocomputing* 429, 110–117. <http://dx.doi.org/10.1016/j.neucom.2020.11.068>, URL <https://www.sciencedirect.com/science/article/pii/S0925231220318749>.
- Zhu, J.Y., Park, T., Isola, P., Efros, A.A., 2020. Unpaired image-to-image translation using cycle-consistent adversarial networks. (*arXiv:1703.10593*), *arXiv:1703.10593[cs]*, URL <http://arxiv.org/abs/1703.10593>.